

Politechnika Wrocławska

Wydział Informatyki i Telekomunikacji

Kierunek: Informatyka
Specjalność: Inżynieria Systemów Informatycznych

PRACA DYPLOMOWA MAGISTERSKA

Projekt i analiza porównawcza wydajności skalowalnych architektur
systemów typu SaaS

Oskar Siemieniuk

Opiekun pracy
dr inż. Konrad Kluwak

Słowa kluczowe: skalowalność, Kubernetes, HPA, mikroserwisy, SaaS, testy obciążeniowe,
Prometheus, autoskalowanie

Spis treści

1	Wstęp teoretyczny	1
1.1	Motywacja i problem badawczy	1
1.2	Hipoteza badawcza, cel i zakres	2
1.2.1	Hipoteza	2
1.2.2	Cel i zadania	2
1.2.3	Zakres	2
1.3	Podstawy teoretyczne	3
1.3.1	Architektury skalowalne: od monolitu do mikroserwisów	3
1.3.2	Wzorce skalowania i autoskalowania	3
1.3.3	Kubernetes i orkiestracja kontenerów	3
1.3.4	Mechanizmy autoskalowania w Kubernetes	3
1.3.5	Model SaaS i multi-tenancy	4
1.4	Przegląd literatury	4
1.4.1	Istniejące badania nad HPA	4
1.4.2	Zidentyfikowane luki badawcze	5
1.5	Struktura pracy	5
2	Przegląd technologii	7
2.1	Google Kubernetes Engine — zarządzany Kubernetes w GCP	7
2.2	GCP LoadBalancer — ekspozycja serwisów	7
2.3	Compute Engine — generator obciążenia	8
2.4	kube-prometheus-stack — stos monitorujący	8
2.4.1	Prometheus	9
2.4.2	Grafana	9
2.4.3	Prometheus Adapter	9
2.5	k6 — narzędzie do testów obciążeniowych	10
2.6	Docker i konteneryzacja	11
2.7	Podsumowanie stosu technologicznego	12
3	Projekt architektury systemu badawczego	13
3.1	Macierz badawcza	13
3.2	Aplikacje badawcze	13
3.2.1	CPU-service — obciążenie CPU-bound	14
3.2.2	IO-service — obciążenie I/O-bound	14
3.2.3	Echo-server — serwis pomocniczy	14
3.2.4	Memory-service — serwis pamięciowy	14
3.3	Strategie HPA	14
3.4	Topologia wdrożeniowa	15
3.5	Infrastruktura monitorująca i eksport danych	16

3.6	Plan eksperymentów	16
3.6.1	Scenariusze testowe	16
3.6.2	Procedura	16
3.6.3	Profile obciążeniowe k6	17
3.6.4	Metryki badawcze	17
3.6.5	Analiza zjawisk przejściowych	17
3.7	Podsumowanie decyzji projektowych	17
4	Implementacja	19
4.1	Struktura repozytorium	19
4.2	Implementacja mikroservisów	19
4.2.1	CPU-service	20
4.2.2	IO-service	20
4.2.3	Echo-server i Memory-service	20
4.3	Konfiguracja Kubernetes	21
4.3.1	Deploymenty	21
4.3.2	HPA — trzy warianty	21
4.3.3	Services	21
4.4	Infrastruktura monitorująca	21
4.4.1	kube-prometheus-stack	21
4.4.2	Prometheus Adapter	21
4.4.3	Grafana — dashboard i eksport	21
4.5	Skrypty automatyzujące	22
4.5.1	run-tests.sh — orkiestracja eksperymentów	22
4.6	Scenariusze testowe k6	22
4.7	Podsumowanie implementacji	22
5	Badania eksperymentalne i analiza wyników	23
5.1	Metodyka badań	23
5.1.1	Środowisko testowe	23
5.1.2	Procedura eksperymentalna	23
5.1.3	Zbierane metryki	24
5.1.4	Metryki badawcze — definicje operacyjne	24
5.2	Scenariusze testowe	25
5.2.1	Scenariusze bazowe	25
5.2.2	Scenariusze CPU-bound	26
5.2.3	Scenariusze I/O-bound	26
5.3	Wyniki eksperymentalne	26
5.3.1	Scenariusze bazowe	27
5.3.2	Scenariusze CPU-bound — porównanie strategii HPA	27
5.3.3	Scenariusze I/O-bound — porównanie strategii HPA	28
5.3.4	Wykresy przebiegu skalowania	28
5.3.5	Analiza zjawisk anomalnych	31
5.4	Analiza wyników	32
5.4.1	Porównanie strategii HPA dla CPU-bound	32
5.4.2	Porównanie strategii HPA dla I/O-bound	33
5.4.3	Efektywność skalowania	34
5.4.4	Analiza zjawisk anomalnych — podsumowanie obserwacji	34
5.5	Dyskusja i interpretacja	35

5.6	Wnioski z badań	35
6	Podsumowanie	37
6.1	Podsumowanie zrealizowanych prac	37
6.2	Weryfikacja hipotezy badawczej	38
6.3	Ograniczenia pracy	39
6.4	Rekomendacje praktyczne	39
6.5	Dalsze kierunki badań	39
6.6	Podsumowanie końcowe	40

Rozdział 1

Wstęp teoretyczny

1.1 Motywacja i problem badawczy

Systemy informatyczne dostarczane w modelu *Software as a Service*, czyli SaaS, stoją przed podwójnym wyzwaniem: muszą jednocześnie obsługiwać zmienne obciążenie generowane przez wielu klientów, zwanych dalej tenantami, oraz utrzymywać przewidywalną wydajność przy racjonalnym koszcie infrastruktury. Współdzielenie zasobów obliczeniowych przez tenantów o różnych profilach aktywności, od lekkiego przeglądania dashboardu po masowy eksport danych, oznacza, że system nie może polegać na statycznym przydziale zasobów. Potrzebuje mechanizmu, który dynamicznie dostosuje liczbę jednostek obliczeniowych do bieżącego zapotrzebowania.

W ekosystemie Kubernetes podstawowym narzędziem realizującym ten cel jest *HorizontalPodAutoscaler*, czyli HPA: kontroler, który na podstawie obserwowanych metryk podejmuje decyzje o dodaniu lub usunięciu replik aplikacji. HPA działa reaktywnie: co 15 sekund porównuje bieżącą wartość wybranej metryki z zadaniem progiem i na tej podstawie oblicza pożądaną liczbę replik. Prostota tego mechanizmu jest jego zaletą operacyjną, ale jednocześnie rodzi pytanie: **czy wybór metryki ma znaczenie dla skuteczności skalowania?**

Intuicja inżynierska podpowiada, że dla aplikacji obliczeniowej, czyli CPU-bound, naturalnym wyborem będzie HPA oparty na zużyciu procesora, a dla aplikacji oczekującej na zewnętrzne odpowiedzi, czyli I/O-bound, będzie to HPA oparty na liczbie żądań. Problem w tym, że w praktyce zespoły wdrożeniowe rzadko analizują profil obciążeniowy swojej aplikacji przed skonfigurowaniem autoskalowania. Domyślna konfiguracja HPA, czyli metryka CPU z progiem 50%, jest stosowana uniwersalnie, niezależnie od charakterystyki serwisu. W środowisku SaaS, gdzie jeden klaster obsługuje dziesiątki tenantów, konsekwencje błędnego doboru metryki mogą być poważne: od niedoskalowania, czyli spadku wydajności dla wszystkich tenantów, po przeskalowanie oznaczające niepotrzebny koszt infrastruktury chmurowej.

Niniejsza praca podejmuje próbę systematycznego rozstrzygnięcia tej kwestii poprzez kontrolowane eksperymenty w środowisku Google Kubernetes Engine, czyli GKE. Zamiast zakładać z góry, która metryka jest najlepsza, badanie traktuje wybór metryki HPA jako zmienną niezależną i mierzy jej wpływ na kluczowe wskaźniki wydajnościowe: przepustowość, opóźnienia, czas reakcji na skalowanie oraz stopę błędów.

1.2 Hipoteza badawcza, cel i zakres

1.2.1 Hipoteza

Hipoteza badawcza pracy brzmi:

Nie istnieje jedna uniwersalna metryka HPA: skuteczność strategii skalowania zależy od charakterystyki obciążeniowej aplikacji, czyli od jej profilu obciążeniowego. Błędny wybór metryki prowadzi do niedoskalowania lub przeskalowania.

Przez „skuteczność” rozumie się zdolność HPA do wykrycia przeciążenia i dodania replik zanim opóźnienia odpowiedzi przekroczą akceptowalny próg, przy jednoczesnym unikaniu niepotrzebnego skalowania w górę gdy obciążenie jest niskie.

1.2.2 Cel i zadania

Celem pracy jest zaprojektowanie, implementacja oraz eksperymentalna walidacja prototypu skalowalnego systemu w modelu SaaS, ze szczególnym uwzględnieniem porównania trzech strategii HPA dla dwóch diametralnie różnych profili obciążeniowych. Cel ten został zrealizowany poprzez następujące zadania:

1. Zaprojektowanie i implementacja dwóch mikroservisów o skrajnie odmiennych charakterystykach: CPU-bound, realizującego przetwarzanie obrazu i obliczenia rekurencyjne, oraz I/O-bound, opartego na łańcuchach asynchronicznych opóźnień symulujących zapytania do zewnętrznych usług.
2. Przeprowadzenie kontrolowanych eksperymentów porównujących trzy strategie HPA: opartą na zużyciu CPU, opartą na zużyciu pamięci oraz opartą na niestandardowej metryce RPS, czyli liczbie żądań na sekundę, dostarczanej przez prometheus-adapter.
3. Analiza porównawcza efektywności strategii skalowania w macierzy badawczej 2×3 , łączącej dwa typy obciążenia z trzema strategiami HPA.
4. Sformułowanie rekomendacji praktycznych dotyczących doboru metryki HPA w zależności od charakterystyki aplikacji.

1.2.3 Zakres

Zakres pracy obejmuje analizę teoretyczną zagadnień związanych z systemami rozproszonymi i autoskalowaniem, budowę autorskiego rozwiązania systemowego w technologii Kubernetes na platformie Google Cloud, przygotowanie środowiska testowego z pełnym stosem monitorującym składającym się z Prometheusa, Grafany i narzędzia k6 oraz przeprowadzenie serii badań eksperymentalnych weryfikujących efektywność zastosowanych technik skalowania.

1.3 Podstawy teoretyczne

1.3.1 Architektury skalowalne: od monolitu do mikroserwisów

Ewolucja architektur systemów rozproszonych, od monolitycznych aplikacji przez architekturę zorientowaną na usługi, czyli SOA, do współczesnego paradygmatu mikroserwisów, była w dużej mierze napędzana potrzebą elastycznego skalowania [12]. W monolicie cała funkcjonalność systemu jest zawarta w jednej jednostce wdrożeniowej; skalowanie odbywa się przez uruchomienie wielu identycznych instancji całej aplikacji, co jest nieefektywne, gdy tylko wybrane komponenty wymagają dodatkowej mocy obliczeniowej. SOA wprowadziło podział na usługi komunikujące się przez Enterprise Service Bus, umożliwiając selektywne skalowanie kosztem złożoności warstwy integracyjnej. Mikroserwisy wynoszą tę ideę na kolejny poziom: każda funkcja biznesowa jest niezależną jednostką wdrożeniową z własnym cyklem życia, stosem technologicznym i, co kluczowe z perspektywy tej pracy, własną polityką skalowania. To właśnie w architekturze mikroserwisowej precyzyjny dobór metryki HPA staje się decyzją o wymiernych konsekwencjach wydajnościowych i kosztowych.

1.3.2 Wzorce skalowania i autoskalowania

Skalowanie systemów realizuje się w trzech podstawowych wymiarach. Skalowanie pionowe, czyli vertical scaling, polega na zwiększaniu zasobów pojedynczej instancji (więcej CPU, więcej pamięci RAM). Jest ograniczone fizycznymi możliwościami sprzętu i nie zwiększa odporności na awarie. Skalowanie poziome, czyli horizontal scaling, zwiększa liczbę instancji obsługujących ruch, co rozkłada obciążenie i podnosi niezawodność, ale wymaga mechanizmów równoważenia obciążenia oraz koordynacji stanu. Trzeci wymiar, skalowanie automatyczne, czyli autoscaling, to dynamiczne dostosowywanie liczby instancji na podstawie obserwowanych metryk, bez interwencji operatora. W systemach chmurowych, gdzie koszty są proporcjonalne do wykorzystania zasobów, autoskalowanie jest nie tyle optymalizacją, co warunkiem koniecznym ekonomicznej eksploatacji.

1.3.3 Kubernetes i orkiestracja kontenerów

Kubernetes, w skrócie K8s, jest otwartą platformą do automatyzacji wdrażania, skalowania i zarządzania aplikacjami kontenerowymi, utrzymywaną przez Cloud Native Computing Foundation, czyli CNCF [2]. Architektura klastra Kubernetes dzieli się na dwie warstwy. Control Plane, zarządzana przez Google w przypadku GKE, obejmuje API server, rozproszony magazyn stanu etcd, scheduler przydzielający Pody do węzłów oraz controller manager, w którym działa m.in. HorizontalPodAutoscaler [17]. Worker Nodes to węzły wykonawcze, na których uruchamiane są Pody, czyli podstawowe jednostki obliczeniowe: grupy jednego lub więcej kontenerów współdzielących przestrzeń sieciową i woluminy. Pody są efemeryczne: mogą być tworzone, niszczone i przenoszone między węzłami, co czyni je naturalnym celem dla mechanizmów autoskalowania.

1.3.4 Mechanizmy autoskalowania w Kubernetes

HorizontalPodAutoscaler, czyli HPA, jest kontrolerem, który automatycznie dostosowuje liczbę replik w Deployment, ReplicaSet lub StatefulSet na podstawie obserwowanych metryk. Algorytm HPA działa według formuły:

$$\text{desiredReplicas} = \left\lceil \text{currentReplicas} \times \frac{\text{currentMetricValue}}{\text{desiredMetricValue}} \right\rceil \quad (1.1)$$

HPA w wersji API `autoscaling/v2`, wykorzystywanej w niniejszej pracy, obsługuje trzy typy metryk: `resource metrics`, czyli metryki zasobów kontenera (takie jak CPU i pamięć), dostarczane przez `metrics-server`; `Pods metrics`, czyli metryki niestandardowe skojarzone z konkretnymi Podami, dostarczane przez zewnętrzne adaptery, np. `prometheus-adapter`; oraz `object metrics`, czyli metryki opisujące obiekty Kubernetes, również dostarczane przez adaptery.

Dodatkowo, HPA implementuje mechanizmy stabilizacyjne zapobiegające zbyt częstym zmianom liczby replik. Okno stabilizacji skalowania w dół, czyli `scale-down stabilization window`, wynosi domyślnie 5 minut; HPA uwzględnia maksymalną wartość replik z tego okna przed podjęciem decyzji o redukcji. Tolerancja wynosząca 10% oznacza, że metryka musi odchylić się o więcej niż 10% od progu, aby wywołać skalowanie. Od wersji `autoscaling/v2` można również definiować polityki ograniczające tempo skalowania, np. maksymalnie 2 nowe repliki na 60 sekund.

VerticalPodAutoscaler, czyli VPA, dostosowuje `resource requests` i `limits` dla kontenerów w Podzie, zmieniając ilość zasobów przydzielonych każdej replice zamiast liczby replik. **KEDA**, czyli Kubernetes Event-Driven Autoscaling, rozszerza HPA o skalowanie na podstawie zdarzeń z zewnętrznych systemów (na przykład długości kolejki Apache Kafka) oraz o możliwość skalowania do zera replik. VPA i KEDA nie są przedmiotem badań w niniejszej pracy, ale stanowią istotne kierunki dalszych prac.

1.3.5 Model SaaS i multi-tenancy

Model *Software as a Service* jest jedną z trzech podstawowych kategorii *cloud computing* (obok *IaaS* i *PaaS*), w której dostawca udostępnia użytkownikom gotową aplikację przez Internet, zarządzając całą infrastrukturą. Kluczową cechą SaaS jest wielodostępność, czyli *multi-tenancy*: architektoniczny wzorzec umożliwiający obsługę wielu klientów przez pojedynczą instancję systemu, przy zachowaniu logicznej izolacji danych i przewidywalnej wydajności dla każdego tenanta.

W kontekście Kubernetes można wyróżnić kilka poziomów izolacji: od najbliższego (*application-level multi-tenancy*, gdzie tenanci są rozróżniani przez nagłówek HTTP i współdzielą ten sam namespace), przez *namespace-per-tenant*, gdzie każdy tenant otrzymuje własny namespace z izolowanymi zasobami, aż po *cluster-per-tenant*, czyli dedykowany klaster dla każdego tenanta, zapewniający najwyższy poziom izolacji przy najwyższym koszcie. Współdzielenie infrastruktury oznacza, że decyzje HPA podejmowane w odpowiedzi na obciążenie jednego tenanta mogą wpływać na wydajność pozostałych; jest to jedno z głównych pytań motywujących niniejsze badanie.

1.4 Przegląd literatury

1.4.1 Istniejące badania nad HPA

Problematyka autoskalowania w Kubernetes była przedmiotem licznych badań w ostatnich latach. Casalicchio i Perciballi [3] przeprowadzili analizę porównawczą domyślnego HPA z KEDA, badając czas reakcji i wykorzystanie zasobów dla obciążeń o różnych profilach. Autorzy wykazali, że KEDA oferuje szybszy czas skalowania przy obciążeniach

zdarzeniowych, ale domyślny HPA jest wystarczający dla obciążeń o stałym charakterze; nie analizowali jednak wpływu konkretnego wyboru metryki w obrębie samego HPA.

Nguyen i Kim zaproponowali predykcyjny algorytm skalowania oparty na sieciach LSTM, czyli Long Short-Term Memory, który przewiduje przyszłe obciążenie i skaluje proaktywnie, redukując liczbę naruszeń SLA o 30% w porównaniu do reaktywnego HPA. Podejście to wymaga jednak znacznych zasobów obliczeniowych na trenowanie modelu i nie zostało zweryfikowane w środowisku wielodostępnym SaaS.

1.4.2 Zidentyfikowane luki badawcze

Na podstawie przeglądu literatury zidentyfikowano następujące luki badawcze, które motywują niniejszą pracę:

1. **Brak systematycznego porównania strategii HPA dla różnych profili obciążeńiowych:** większość badań testuje HPA na jednorodnym obciążeniu (wyłącznie CPU-bound lub wyłącznie I/O-bound), nie analizując interakcji między profilem obciążeniowym a wyborem metryki.
2. **Ograniczona analiza HPA Memory:** strategia oparta na zużyciu pamięci jest rzadko badana empirycznie, ponieważ zakłada się z góry, że pamięć w większości aplikacji nie koreluje z obciążeniem. Niniejsza praca dostarcza danych eksperymentalnych weryfikujących to założenie.
3. **Brak badań łączących HPA z modelem SaaS:** istniejące badania nad multi-tenancy koncentrują się na izolacji danych i bezpieczeństwie, pomijając wpływ wielodostępności na efektywność autoskalowania.
4. **Brak systematycznej analizy zjawisk przejściowych:** istniejące publikacje koncentrują się na metrykach zagregowanych (średnim RPS czy średnim opóźnieniu), pomijając analizę zjawisk takich jak opóźnienie reakcji HPA, czyli HPA Lag, opóźnienie zimnego startu Podów, czyli Cold Start Latency, czy kaskadowe błędy przy CPU throttlingu. Niniejsza praca identyfikuje i dokumentuje te zjawiska na podstawie danych eksperymentalnych.

1.5 Struktura pracy

Praca składa się z sześciu rozdziałów. Rozdział 2 zawiera szczegółowy przegląd technologii wykorzystanych w projekcie, ze szczególnym uwzględnieniem komponentów, które nie zostały omówione w części teoretycznej: Google Kubernetes Engine jako zarządzanej platformy orkiestracji, stosu monitorującego kube-prometheus-stack, narzędzia do testów obciążeniowych k6 oraz prometheus-adapter jako mostka między metrykami aplikacyjnymi a API HPA. Rozdział 3 przedstawia projekt architektury systemu badawczego, w tym macierz badawczą 2×3 , specyfikację mikroservisów, trzy warianty HPA oraz plan eksperymentów. Rozdział 4 dokumentuje implementację: strukturę repozytorium, kod mikroservisów, konfigurację Kubernetes i infrastrukturę monitorującą. Rozdział 5 zawiera wyniki badań eksperymentalnych przeprowadzonych na klastrze GKE wraz z ich analizą i interpretacją. Rozdział 6 zamyka pracę podsumowaniem, weryfikacją hipotezy, rekomendacjami praktycznymi oraz wskazaniem ograniczeń i dalszych kierunków badań.

Rozdział 2

Przegląd technologii

W rozdziale tym omówiono technologie wykorzystane w projekcie badawczym, które nie zostały szczegółowo opisane w części teoretycznej (Rozdział 1). Nacisk położono na komponenty specyficzne dla środowiska Google Cloud Platform, czyli GCP: zarządzany klastrer GKE, równoważenie obciążenia przez GCP LoadBalancer, maszynę Compute Engine jako generator obciążenia testowego oraz stos monitorujący kube-prometheus-stack z prometheus-adapter jako mostkiem między metrykami aplikacyjnymi a API HPA.

2.1 Google Kubernetes Engine — zarządzany Kubernetes w GCP

Google Kubernetes Engine, czyli GKE, jest zarządzaną usługą Kubernetes oferowaną przez Google Cloud Platform, certyfikowaną przez CNCF pod kątem pełnej zgodności API [7]. W odróżnieniu od samodzielnie administrowanego klastra, GKE automatycznie zarządza warstwą Control Plane; użytkownik nie ma bezpośredniego dostępu do komponentów takich jak etcd czy kube-apiserver, co eliminuje konieczność ich ręcznej konfiguracji i utrzymania.

W projekcie wykorzystano klastrer GKE w konfiguracji VPC-native na maszynach typu `e2-standard-4` (4 vCPU, 16 GB RAM). Wybór GKE jako platformy badawczej podyktowany był czterema czynnikami. Po pierwsze, pełna zgodność API gwarantuje, że wszystkie manifesty (Deployment, HPA, Service) działają identycznie jak na dowolnym klastrze zgodnym z CNCF. Po drugie, integracja z GCP LoadBalancer umożliwia bezpośrednią ekspozycję serwisów aplikacyjnych na zewnętrzne adresy IP bez potrzeby tunelowania. Po trzecie, maszyna generująca obciążenie testowe (Compute Engine) znajduje się w tej samej sieci VPC co klastrer, co zapewnia niskie opóźnienie sieciowe ($\sim 1\text{--}5$ ms). Po czwarte, GKE cluster autoscaler automatycznie dostosowuje liczbę węzłów do zapotrzebowania, umożliwiając testy z dużą liczbą Podów bez ręcznej interwencji.

2.2 GCP LoadBalancer — ekspozycja serwisów

W środowisku GKE serwisy typu LoadBalancer są automatycznie integrowane z infrastrukturą równoważenia obciążenia GCP [5]. Po utworzeniu obiektu Service typu LoadBalancer, kontroler GKE (cloud controller manager) tworzy regułę forwarding rule w GCP, a platforma przydziela publiczny adres IP (regionalny, statyczny lub efemeryczny). Ruch przychodzący na ten adres jest dystrybuowany do Podów przez kube-proxy na węzłach.

W projekcie wszystkie serwisy aplikacyjne są eksponowane przez LoadBalancer, co umożliwia maszynie generującej obciążenie testowe bezpośrednie wysyłanie żądań HTTP. Rozwiązanie to jest istotnie lepsze od powszechnie stosowanego w środowiskach deweloperskich mechanizmu `kubectl port-forward`, który tuneluje ruch przez pojedyncze połączenie WebSocket do API server; przy dużym obciążeniu tworzy to wąskie gardło i dodaje nienaturalne opóźnienia. GCP LoadBalancer eliminuje te ograniczenia, zapewniając bezpośrednią ścieżkę sieciową między generatorem obciążenia a Podami, co odzwierciedla rzeczywiste warunki produkcyjne.

2.3 Compute Engine — generator obciążenia

Google Compute Engine (GCE) to usługa maszyn wirtualnych GCP [6]. W projekcie wykorzystano dedykowaną maszynę, nazwaną **k6-generator**, jako wyłącznego hosta dla narzędzia k6, generującego obciążenie testowe. Umieszczenie generatora na osobnej maszynie w tej samej sieci VPC co klaster GKE rozwiązuje dwa problemy: zapobiega zakłócaniu wyników testów przez konkurencję o zasoby (generator i badane serwisy nie dzielą CPU ani pamięci) oraz zapewnia realistyczne, niskie opóźnienia sieciowe między nadawcą a odbiorcą ruchu.

Zamiast konfiguracji DNS, która wymagałaby rejestracji domen i propagacji rekordów, skrypty k6 wykorzystują mechanizm `options.hosts` do bezpośredniego mapowania nazw serwisów na adresy IP GCP LoadBalancer. Mechanizm ten działa na poziomie biblioteki HTTP k6 i podmienia nagłówki `Host` oraz docelowy adres IP dla każdego żądania.

```

1 export const options = {
2   hosts: {
3     'cpu-service': '34.118.71.122',
4     'io-service': '35.204.93.45',
5   },
6   scenarios: { ... }
7 };

```

Listing 2.1 Mapowanie hostów w opcjach skryptu k6

2.4 kube-prometheus-stack — stos monitorujący

kube-prometheus-stack jest pakietem Helm, czyli Helm chart, [9] rozwijanym przez społeczność prometheus-community, który instaluje kompletny stos monitorujący dla Kubernetes [14]. W skład pakietu wchodzi: Prometheus (system zbierania i przechowywania metryk w postaci szeregów czasowych), Grafana (platforma wizualizacji danych), kube-state-metrics (komponent generujący metryki o stanie obiektów Kubernetes), node-exporter (komponent zbierający metryki systemowe z węzłów) oraz Prometheus Operator (kontroler zarządzający cyklem życia instancji Prometheusa) [15].

Wybór instalacji przez Helm, zamiast ręcznego wdrażania pojedynczych manifestów YAML, jest podyktowany automatyczną konfiguracją: ServiceMonitor i PodMonitor automatycznie wykrywają źródła metryk na podstawie etykiet, bez konieczności ręcznej edycji pliku `prometheus.yml`. Ponadto, Grafana otrzymuje zestaw gotowych dashboardów dla Kubernetes, a Prometheus Operator zarządza aktualizacjami i skalowaniem.

Pakiet instaluje się jednym zestawem poleceń:

```

1 helm repo add prometheus-community \

```

```
2 https://prometheus-community.github.io/helm-charts
3 helm repo update
4 helm install monitoring prometheus-community/kube-prometheus-stack \
5   --namespace magisterka --create-namespace \
6   --set prometheus.prometheusSpec.serviceMonitorSelectorNilUsesHelmValues=
   false \
7   --set grafana.service.type=LoadBalancer
```

Listing 2.2 Instalacja kube-prometheus-stack przez Helm

Parametr `grafana.service.type=LoadBalancer` eksponuje Grafanę publicznie przez GCP LoadBalancer, umożliwiając obserwację eksperymentów w czasie rzeczywistym z dowolnej przeglądarki.

2.4.1 Prometheus

Prometheus jest otwartoźródłowym systemem monitorowania, pierwotnie opracowanym w SoundCloud, obecnie inkubowanym przez CNCF [1]. Jego architektura opiera się na modelu pull: Prometheus aktywnie pobiera, czyli scrapuje, metryki z aplikacji poprzez HTTP w konfigurowalnych interwałach, w projekcie co 15 sekund. Zebrane dane są przechowywane w natywnej bazie szeregów czasowych (TSDB) z etykietami umożliwiającymi filtrowanie i agregację. Język zapytań PromQL pozwala na zaawansowane operacje: agregacje, operacje na zakresach czasowych oraz predykcję.

W projekcie Prometheus pełni podwójną rolę. Zbiera metryki infrastrukturalne (CPU, RAM, liczba replik, stan HPA) poprzez kube-state-metrics i node-exporter oraz metryki aplikacyjne z mikroserwisów (RPS, czas przetwarzania, liczba aktywnych zapytań) z endpointów `/metrics`. Jednocześnie udostępnia te dane przez API dla prometheus-adapter, który transformuje zapytania PromQL na format zrozumiały dla HPA Kubernetes.

2.4.2 Grafana

Grafana jest platformą do wizualizacji danych, integrującą się z wieloma źródłami, w tym z Prometheusem. W projekcie wykorzystano autorski dashboard „Magisterka – System Overview” zawierający 16 paneli odświeżanych co 10 sekund, pokazujących kluczowe metryki: RPS, zużycie CPU i pamięci per Pod, percentyle opóźnień (p50, p95, p99), liczbę replik oraz stan HPA. Dashboard jest dostępny publicznie przez GCP LoadBalancer.

Dla każdego scenariusza testowego z Grafany eksportowanych jest 5 zestawów danych CSV: liczba replik w czasie, zużycie CPU per Pod w mCPU, zużycie pamięci per Pod w MiB, przepustowość w RPS oraz percentyle opóźnień w ms. Dane te, wraz z wynikami k6, stanowią kompletny zbiór danych badawczych dla analizy w Rozdziale 5.

2.4.3 Prometheus Adapter

Prometheus Adapter (wersja v0.12.0) jest komponentem implementującym Kubernetes Custom Metrics API [11]. Jego rola polega na rejestracji jako APIService w Kubernetes (rozszerzając API o ścieżki `/apis/custom.metrics.k8s.io/`), pobieraniu metryk z Prometheusa poprzez zapytania PromQL oraz serwowaniu tych metryk w formacie oczekiwanym przez HPA.

Konfiguracja adaptera jest przechowywana w ConfigMap (`prometheus-adapter-config`) i definiuje reguły mapowania metryk Prometheusa na metryki Kubernetes. W projekcie zdefiniowano cztery reguły: dwie dla metryki RPS (przeliczanej przez funkcję `rate()` [1m]

z liczników żądań HTTP) oraz dwie dla metryki liczby żądań w toku (in-flight), po jednej parze dla cpu-service i io-service.

```

1 rules:
2   - seriesQuery: 'cpu_service_http_requests_total{namespace!="",pod!=""}'
3     resources:
4       overrides:
5         namespace: {resource: "namespace"}
6         pod: {resource: "pod"}
7     name:
8       matches: "cpu_service_http_requests_total"
9       as: "http_requests_per_second"
10    metricsQuery: 'sum(rate(cpu_service_http_requests_total{<<.
      LabelMatchers>>}[1m])) by (<<.GroupBy>>)'
11  - seriesQuery: 'io_service_http_requests_total{namespace!="",pod!=""}'
12    resources:
13      overrides:
14        namespace: {resource: "namespace"}
15        pod: {resource: "pod"}
16    name:
17      matches: "io_service_http_requests_total"
18      as: "http_requests_per_second"
19    metricsQuery: 'sum(rate(io_service_http_requests_total{<<.LabelMatchers
      >>}[1m])) by (<<.GroupBy>>)'
20  # ... (pozostae dwie reguły dla metryk in-flight, analogiczne)

```

Listing 2.3 Konfiguracja Prometheus Adapter — wybrane reguły mapowania metryk

2.5 k6 — narzędzie do testów obciążeniowych

k6 (Grafana k6) jest otwartoźródłowym narzędziem do testów obciążeniowych, napisanym w Go z interfejsem skryptowym w JavaScript (ES6) [8]. Pojedyncza instancja k6 jest w stanie generować setki wirtualnych użytkowników, czyli VU (Virtual Users) bez potrzeby architektury rozproszonej, co czyni ją odpowiednią do testów średniej skali bez dodatkowej infrastruktury.

W projekcie zdefiniowano trzy skrypty k6 odpowiadające profilom obciążeniowym i scenariuszom badawczym. Skrypt `cpu-load-test.js` zawiera dwa scenariusze: Fibonacciego (ramp-up od 1 do 50 VU przez 60 sekund, następnie stałe obciążenie 50 VU przez 120 sekund) oraz przetwarzanie obrazu (stałe 10 VU wysyłających losowo generowane obrazy PNG do endpointu `POST /process`). Skrypt `io-load-test.js` zawiera trzy scenariusze: zapytania ze stałym opóźnieniem z ramp-up od 1 do 100 VU, test obciążeniowy z rosnącym opóźnieniem od 50 do 2000 ms oraz wywołania zewnętrzne. Skrypt `combined-scenario.js` uruchamia trzy scenariusze równoległe: Fibonacciego (30 VU), przetwarzanie obrazu (5 VU) oraz zapytania IO (80 VU).

Każdy skrypt definiuje progi akceptacji, czyli thresholds: percentyl p95 czasu odpowiedzi poniżej 5000 ms dla scenariuszy CPU oraz poniżej 10000 ms dla scenariuszy IO, a także dopuszczalną stopę błędów HTTP (< 1% dla CPU, < 5% dla IO). Przekroczenie tych progów oznacza niepowodzenie testu.

2.6 Docker i konteneryzacja

Każdy mikroservis w projekcie jest konteneryzowany za pomocą Docker [13]. Wszystkie Dockerfile opierają się na wspólnym wzorcu: obraz bazowy `python:3.12-slim`, wybrany ze względu na minimalny rozmiar; instalacja zależności przez `pip install --no-cache-dir`; aplikacja jako pojedynczy plik `main.py`; uruchomienie przez Uvicorn z parametrami `--host 0.0.0.0 --port 8000`.

Obrazy są budowane lokalnie i przesyłane do GCP Artifact Registry [4], zarządzanego repozytorium obrazów kontenerowych Google Cloud, pełniącego w projekcie rolę rejestru obrazów dla GKE. W przeciwieństwie do publicznego Docker Hub, Artifact Registry zapewnia niskie opóźnienie pobierania obrazów w ramach tej samej sieci VPC, kontrolę dostępu przez IAM oraz automatyczne skanowanie podatności. GKE pobiera obrazy automatycznie podczas wdrażania na podstawie pola `image` w Deployment. W projekcie nie wykorzystuje się Docker Compose do orkiestracji testów; wszystkie eksperymenty są przeprowadzane bezpośrednio na klastrze GKE, co zapewnia zgodność środowiska testowego z produkcyjnym. Plik `docker-compose.yaml` znajdujący się w repozytorium służy wyłącznie do opcjonalnego, szybkiego uruchomienia serwisów w środowisku deweloperskim.

2.7 Podsumowanie stosu technologicznego

Warstwa	Technologia	Rola w projekcie
Język / Framework	Python 3.12 + FastAPI [16]	Implementacja logiki mikro-serwisów: <code>cpu-service</code> , <code>io-service</code> , <code>echo-server</code> , <code>memory-service</code>
Serwer ASGI	Uvicorn	Obsługa żądań HTTP z wykorzystaniem pętli zdarzeń asynchronicznych
Konteneryzacja	Docker [13]	Budowa lekkich obrazów kontenerowych na bazie <code>python:3.12-slim</code>
Rejestr obrazów	GCP Artifact Registry [4]	Prywatny rejestr obrazów kontenerowych w sieci VPC; źródło dla GKE
Orkiestracja	GKE (Google Kubernetes Engine) [7]	Zarządzanie klastrem, harmonogramowanie Podów, <i>cluster autoscaler</i>
Sieć	GCP VPC + LoadBalancer [5]	Komunikacja wewnętrzna i ekspozycja serwisów na zewnętrzne adresy IP
Monitoring	kube-prometheus-stack [14] (Helm [9])	Zbieranie metryk (Prometheus), wizualizacja (Grafana), metryki stanu klastra (kube-state-metrics)
Custom Metrics	prometheus-adapter v0.12.0 [11]	Translacja zapytań PromQL na Custom Metrics API Kubernetes dla HPA
Testy obciążeniowe	k6 v0.48+ [8]	Generowanie ruchu HTTP i pomiar metryk wydajnościowych (RPS, percentyle opóźnień)
Generator obciążenia	GCP Compute Engine [6]	Dedykowana maszyna wirtualna (<code>k6-generator</code>) izolująca obciążenie testowe od klastra

Tabela 2.1 Podsumowanie stosu technologicznego projektu wraz z rolami poszczególnych komponentów w systemie badawczym

Rozdział 3

Projekt architektury systemu badawczego

Rozdział niniejszy opisuje projekt systemu badawczego, jego komponenty, strategię skalowania oraz plan eksperymentów. Wszystkie decyzje projektowe podporządkowano celowi badawczemu: porównaniu skuteczności trzech strategii HorizontalPodAutoscaler dla dwóch diametralnie różnych profili obciążeniowych, w kontrolowanym środowisku GKE.

3.1 Macierz badawcza

Podstawą metodologii badawczej jest macierz 2×3 obejmująca dwie aplikacje o odmiennych profilach obciążeniowych i trzy strategie HPA:

Profil aplikacji	HPA CPU (50%)	HPA Memory (70%)	HPA Custom RPS (100/Pod)
CPU-bound (cpu-service)	cpu-cpu	cpu-memory	cpu-custom
I/O-bound (io-service)	io-cpu	io-memory	io-custom

Tabela 3.1 Macierz badawcza 2×3 — identyfikatory scenariuszy testowych

Każda z sześciu kombinacji jest testowana w odrębnym scenariuszu. Macierz umożliwia izolowanie wpływu pojedynczej zmiennej niezależnej (strategii HPA) przy kontrolowanym profilu obciążeniowym. Oprócz scenariuszy macierzy zdefiniowano dwa scenariusze bazowe: **baseline-cpu** oraz **baseline-io**, w których serwisy działają ze stałą liczbą replik, dokładnie 1, bez HPA; służą one jako punkt odniesienia do oceny, czy autoskalowanie w ogóle poprawia wydajność w stosunku do konfiguracji statycznej.

3.2 Aplikacje badawcze

System składa się z trzech mikroservisów: dwóch podlegających badaniu (cpu-service, io-service) oraz jednego pomocniczego (echo-server). Każdy mikroservis jest niezależną aplikacją FastAPI (Python 3.12), konteneryzowaną za pomocą Docker i wdrażaną jako Deployment w Kubernetes. Wspólnymi cechami wszystkich serwisów są: automatyczna instrumentacja Prometheus przez bibliotekę **prometheus-fastapi-instrumentator**, endpoint health check **GET /health** wykorzystywany przez sondy Kubernetes oraz middleware CORS dla wygodny testowania.

3.2.1 CPU-service — obciążenie CPU-bound

CPU-service implementuje dwa endpointy obciążające procesor. `POST /process` przyjmuje plik graficzny, wykonuje sekwencję operacji obciążających procesor z użyciem biblioteki Pillow: konwersję do skali szarości, filtry BLUR, SHARPEN, FIND_EDGES, ekstrakcję kanałów RGB i resize, a następnie oblicza entropię Shannona dla każdego kanału i zwraca statystyki przetwarzania. `GET /fibonacci` oblicza n -tą liczbę Fibonacciego rekurencyjnym algorytmem o złożoności $O(2^n)$; parametr n jest konfigurowalny, domyślnie 25. Obie operacje są czysto CPU-intensywne, nie wykonują operacji sieciowych ani dyskowych poza odbiorem żądania i wysłaniem odpowiedzi. Dzięki temu zużycie CPU silnie koreluje z liczbą żądań.

3.2.2 IO-service — obciążenie I/O-bound

IO-service symuluje operacje wejścia-wyjścia przez dwa endpointy. `GET /query` wykonuje łańcuch asynchronicznych opóźnień: sekwencję wywołań `asyncio.sleep()` z konfigurowalnym opóźnieniem bazowym (domyślnie 100 ms), liczbą kroków (domyślnie 3) oraz losowym rozrzutem $\pm 20\%$. Procesor pozostaje bezczynny podczas oczekiwania; zużycie CPU nie koreluje z liczbą żądań, co czyni serwis czysto I/O-bound. `GET /upstream` przekazuje żądanie do echo-server i zwraca odpowiedź z dodatkowym opóźnieniem. Serwis wykorzystuje `asyncio.Semaphore(50)` do ograniczenia liczby współbieżnych zapytań.

3.2.3 Echo-server — serwis pomocniczy

Echo-server jest minimalistycznym serwisem pełniącym rolę zależności zewnętrznej dla io-service. Udostępnia endpoint `POST /echo`, który zwraca kopię otrzymanego żądania, umożliwiając weryfikację poprawnego przekazywania danych w scenariuszu wywołań zewnętrznych. Echo-server jest wyłączony z macierzy badawczej.

3.2.4 Memory-service — serwis pamięciowy

W ramach rozwoju projektu zaimplementowano również memory-service, serwis z pamięciową bazą klucz-wartość symulujący obciążenie typu memory-bound. Nie został on włączony do głównej macierzy badawczej; stanowi rozszerzenie do potencjalnych przyszłych badań.

3.3 Strategie HPA

Dla każdej aplikacji badawczej zdefiniowano trzy warianty HorizontalPodAutoscaler, różniące się wyłącznie typem metryki. Wszystkie HPA współdzielą parametry konfiguracyjne: `minReplicas=1`, `maxReplicas=30`, `API autoscaling/v2`. Wartość `minReplicas=1` oznacza, że HPA może skalować od pojedynczej repliki, co odpowiada początkowej liczbie replik zdefiniowanej w Deploymentach.

HPA CPU monitoruje średnie zużycie procesora jako procent `resource request`, a nie jako procent limitu procesora kontenera. Próg: 50%. Przykładowo dla `io-service`, gdzie `requests.cpu=100 m`, próg 50% odpowiada wartości około 50 mCPU. Jest to strategia najczęściej stosowana w ekosystemie Kubernetes; jej skuteczność zależy od tego, czy zużycie CPU koreluje z obciążeniem aplikacji.

HPA Memory monitoruje średnie zużycie pamięci jako procent resource request. Próg: 70%. Strategia ta jest rzadziej stosowana, ponieważ większość aplikacji alokuje pamięć statycznie przy uruchomieniu; metryka pamięci nie odzwierciedla wówczas bieżącego obciążenia. Hipoteza badawcza przewiduje, że HPA Memory będzie nieskuteczne dla obu profili obciążeniowych.

HPA Custom RPS wykorzystuje niestandardową metrykę `http_requests_per_second`, czyli liczbę żądań HTTP na sekundę na Pod, dostarczaną przez prometheus-adapter. Próg: 100 żądań na sekundę na Pod. Metryka RPS jest bezpośrednio powiązana z obciążeniem żadaniami niezależnie od tego, czy żądania są CPU-intensywne czy I/O-intensywne, co czyni tę strategię potencjalnie uniwersalną.

```
1 apiVersion: autoscaling/v2
2 kind: HorizontalPodAutoscaler
3 metadata:
4   name: cpu-service-custom
5 spec:
6   scaleTargetRef:
7     apiVersion: apps/v1
8     kind: Deployment
9     name: cpu-service
10  minReplicas: 1
11  maxReplicas: 30
12  metrics:
13    - type: Pods
14      pods:
15        metric:
16          name: http_requests_per_second
17        target:
18          type: AverageValue
19          averageValue: "100"
```

Listing 3.1 Przykładowa konfiguracja HPA Custom RPS

3.4 Topologia wdrożeniowa

Wszystkie komponenty systemu badawczego działają w środowisku Google Cloud Platform, w pojedynczej sieci VPC obejmującej klaster GKE i maszynę Compute Engine (k6-generator). W klastrze, w namespace `magisterka`, znajdują się: trzy Deployments aplikacyjne (`cpu-service`, `io-service`, `echo-server`), trzy Serwisy typu LoadBalancer kierujące ruch na port 80 do targetPort 8000 odpowiedniego Deploymentu oraz sześć HPA (po trzy na serwis badawczy). Warstwę monitorującą tworzą kube-prometheus-stack (Prometheus + Grafana + kube-state-metrics + node-exporter) oraz prometheus-adapter.

Ruch z k6 do serwisów jest kierowany przez GCP LoadBalancer na podstawie adresów IP mapowanych w opcjach `hosts` skryptów testowych. Opóźnienie sieciowe między generatorem obciążenia a Podami wynosi $\sim 1\text{--}5$ ms, co wynika z komunikacji wewnątrz tej samej strefy VPC. Tabela 3.2 przedstawia przydział zasobów dla poszczególnych serwisów; różnice odzwierciedlają odmienne profile obciążeniowe.

Serwis	CPU request	CPU limit	Memory request / limit
cpu-service	250 m	1000 m	256 Mi / 512 Mi
io-service	100 m	500 m	128 Mi / 256 Mi
echo-server	50 m	200 m	64 Mi / 128 Mi
memory-service	200 m	500 m	256 Mi / 1024 Mi

Tabela 3.2 Zasoby przydzielone serwisom

3.5 Infrastruktura monitorująca i eksport danych

Stos monitorujący oparty na kube-prometheus-stack (Helm) dostarcza Prometheusa (scrape co 15s) oraz Grafanę z autorskim dashboardem „Magisterka – System Overview”. Dashboard jest eksponowany publicznie przez GCP LoadBalancer. Aplikacje badawcze są automatycznie wykrywane przez ServiceMonitor dzięki adnotacjom `prometheus.io/scrape: "true"` na Podach.

Dla każdego z 8 scenariuszy testowych eksportowanych jest 5 zestawów danych CSV z Grafany: liczba replik HPA w czasie, zużycie CPU per Pod w mCPU, zużycie pamięci per Pod w MiB, przepustowość RPS w czasie oraz percentyle opóźnień (p50, p95, p99). Dane te, wraz z wynikami k6 w formacie JSON i podsumowaniu tekstowym, stanowią kompletny zbiór dla każdego scenariusza.

Prometheus Adapter implementuje Custom Metrics API, umożliwiając HPA odczytywanie metryk aplikacyjnych. Konfiguracja definiuje cztery reguły mapowania: po dwie dla `cpu-service` i `io-service`; każda para obejmuje metrykę RPS (przez `rate()` [1m] z liczników żądań) oraz metrykę liczby żądań w toku.

3.6 Plan eksperymentów

3.6.1 Scenariusze testowe

Eksperyment obejmuje 8 scenariuszy uruchamianych sekwencyjnie: dwa bazowe (`baseline-cpu` i `baseline-io`) oraz sześć z macierzy badawczej: `cpu-cpu`, `cpu-memory`, `cpu-custom`, `io-cpu`, `io-memory` i `io-custom`.

3.6.2 Procedura

Każdy scenariusz jest wykonywany według ściśle określonej procedury, w całości zautomatyzowanej przez skrypt `run-tests.sh` działający na maszynie k6-generator. Procedura obejmuje: przełączenie HPA na odpowiednią strategię (to znaczy usunięcie wszystkich istniejących HPA dla docelowego serwisu i nałożenie pliku HPA odpowiadającego testowanej strategii), reset Deploymentu do 1 repliki z oczekiwaniem na gotowość, uruchomienie testu k6 z ruchem kierowanym na adres IP GCP LoadBalancer, zapis wyników do katalogu `results/<scenario>/`. Scenariusz jest powtarzany 3–5-krotnie. Po każdym scenariuszu następuje cooldown: skalowanie obu serwisów do 1 repliki, usunięcie wszystkich HPA, odczekanie 120 sekund na stabilizację klastra.

3.6.3 Profile obciążeniowe k6

Dla CPU-bound wykorzystywany jest skrypt `cpu-load-test.js` z dwoma scenariuszami: Fibonacci (ramp-up od 1 do 50 VU przez 60s, następnie steady przez 120s) oraz przetwarzanie obrazu (stałe 10 VU przez 180s). Dla I/O-bound wykorzystywany jest skrypt `io-load-test.js` z trzema scenariuszami: stałe zapytania (ramp-up od 1 do 100 VU przez 120s, steady 180s), stres-test (stałe 30 VU, delay rosnący od 50 do 2000 ms) oraz wywołania zewnętrzne (20 VU z parametrem `external_call=true`). Skrypt `combined-scenario.js` uruchamia trzy scenariusze równolegle: Fibonacci z 30 VU, przetwarzanie obrazu z 5 VU i zapytania IO z 80 VU.

3.6.4 Metryki badawcze

Dla każdego scenariusza zbierane są: opóźnienia odpowiedzi wyrażone percentylami p50, p95 i p99 w ms, przepustowość w RPS, czas do skalowania (liczony od przekroczenia progu HPA do osiągnięcia docelowej liczby replik), efektywność skalowania rozumiana jako stosunek względnej zmiany przepustowości do względnej zmiany liczby replik, stopa błędów mierzona odsetkiem żądań z kodem HTTP ≥ 400 , liczba replik w czasie oraz zużycie CPU i pamięci na Pod.

3.6.5 Analiza zjawisk przejściowych

Szczególne uwagi w trakcie eksperymentów zostanie poświęcona trzem zjawiskom przejściowym, które mają istotny wpływ na zachowanie systemów w środowisku chmurowym.

HPA Lag oznacza opóźnienie $\sim 60\text{--}90$ sekund między przekroczeniem progu metryki a faktycznym utworzeniem i osiągnięciem gotowości przez nowe Pody. Wynika ono z sumy: czasu scrapowania metryk przez Prometheusa (15s), czasu odświeżenia HPA (15s), czasu tworzenia Podów (2–5s) oraz Cold Start Latency.

Cold Start Latency to czas potrzebny nowo utworzonemu Podowi na osiągnięcie gotowości do obsługi ruchu, obejmujący pobranie obrazu z rejestru kontenerów, uruchomienie kontenera, inicjalizację aplikacji i zaliczenie readiness probe. Łącznie 15–30 sekund, w trakcie których LoadBalancer może kierować ruch do niedojrzałego Poda, powodując błędy HTTP 502/503.

CPU Throttling Cascade to zjawisko kaskadowej degradacji wydajności, gdy Pody osiągają limit CPU podczas gwałtownego wzrostu ruchu. Mechanizm CFS bandwidth control w jądrze Linux ogranicza czas procesora, co powoduje kolejkowanie żądań, wzrost opóźnień, timeouty w k6 i dalsze blokowanie wątków roboczych Uvicorn. Degradacja trwa do momentu osiągnięcia gotowości przez nowe Pody utworzone przez HPA.

Wszystkie trzy zjawiska są identyfikowane i mierzone na podstawie korelacji między danymi z eksportów CSV z Grafany: HPA Lag jako przesunięcie czasowe między krzywą RPS a krzywą liczby replik, Cold Start Latency jako czas od pojawienia się nowej repliki do spadku opóźnień P99, a CPU Throttling Cascade przez jednoczesne wystąpienie wysokiego zużycia CPU, gwałtownego wzrostu opóźnień i wzrostu stopy błędów.

3.7 Podsumowanie decyzji projektowych

Zaprojektowany system badawczy realizuje pięć celów architektonicznych. **Izolacja zmiennych**: każdy scenariusz testuje dokładnie jedną kombinację profil-strategia, przy wyizolowanym generatorze obciążenia i ustabilizowanym klastrze. **Powtarzalność**: każdy sce-

nariusz jest uruchamiany wielokrotnie, co umożliwia analizę statystyczną. **Automatyzacja:** cały proces (przełączanie HPA, reset Deploymentów, uruchamianie k6, eksport CSV z Grafany) jest zautomatyzowany przez skrypt `run-tests.sh`. **Odtwarzalność:** wszystkie manifesty, skrypty k6 i konfiguracje są przechowywane w repozytorium. **Realizm wdrożeniowy:** wykorzystanie GCP LoadBalancer zamiast port-forward oraz dedykowanej maszyny Compute Engine dla k6 odzwierciedla rzeczywiste warunki produkcyjne.

Rozdział 4

Implementacja

Rozdział niniejszy dokumentuje implementację systemu badawczego. Wszystkie listingi odwołują się do rzeczywistych plików w repozytorium projektu.

4.1 Struktura repozytorium

Kod projektu zorganizowano w następującej strukturze:

```
magisterka/
  apps/
    cpu-service/      # CPU-bound microservice
    io-service/       # I/O-bound microservice
    echo-server/      # Auxiliary echo service
    memory-service/   # Memory-bound microservice (extension)
  k8s/
    base/             # Namespace
    cpu-service/      # Deployment + Service + HPA x3
    io-service/       # Deployment + Service + HPA x3
    echo-service/     # Deployment + Service
    memory-service/   # Deployment + Service + HPA
    monitoring/       # Prometheus Adapter config
  monitoring/
    dashboards/       # Grafana dashboard JSON
  scripts/
    run-tests.sh      # Test orchestration (8 scenarios)
  tests/
    k6/               # k6 load test scripts
  docs/
    prd.md            # Product Requirements Document
  docker-compose.yaml # Optional local development (not used in research)
```

4.2 Implementacja mikroservisów

Każdy mikroservis jest niezależną aplikacją FastAPI w wersji Python 3.12 [16], konteneryzowaną przez Docker i uruchamianą przez Uvicorn. Wszystkie serwisy współdzielą

wspólny wzorzec implementacyjny: obraz bazowy `python:3.12-slim`, instalację zależności przez `pip install --no-cache-dir`, automatyczną instrumentację Prometheus przez `prometheus-fastapi-instrumentator` (metryki HTTP na endpointcie `/metrics`) oraz endpoint `GET /health` wykorzystywany przez sondy Kubernetes.

```

1 FROM python:3.12-slim
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install --no-cache-dir -r requirements.txt
5 COPY main.py .
6 EXPOSE 8000
7 CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

```

Listing 4.1 Dockerfile — wzorzec wspólny dla wszystkich serwisów

Obrazy są budowane lokalnie i przesyłane do GCP Artifact Registry, skąd GKE pobiera je automatycznie podczas wdrażania na podstawie pola `image` w Deployment.

4.2.1 CPU-service

CPU-service implementuje dwa endpointy obciążające procesor. `POST /process` akceptuje obraz w formacie multipart form-data i wykonuje potok przetwarzania CPU-intensywnego z biblioteką Pillow: konwersja do skali szarości, filtry BLUR, SHARPEN i FIND_EDGES, ekstrakcja kanałów RGB z resize. Po przetworzeniu, dla każdego kanału obliczana jest entropia Shannona:

$$H = - \sum_{i=0}^{255} p_i \log_2(p_i), \quad \text{gdzie } p_i = \frac{\text{histogram}[i]}{\text{total_pixels}} \quad (4.1)$$

`GET /fibonacci` oblicza n -tą liczbę Fibonacciego rekurencyjnym algorytmem o złożoności $O(2^n)$, co gwarantuje wysokie zużycie CPU przy niewielkiej liczbie żądań.

Serwis wystawia również niestandardowe metryki Prometheus: `cpu_service_images_processed_total` (Counter) oraz `cpu_service_processing_duration_seconds` (Histogram).

4.2.2 IO-service

IO-service symuluje operacje I/O przez dwa endpointy. `GET /query` wykonuje sekwencję wywołań `asyncio.sleep()` z konfigurowalnym opóźnieniem (parametr `delay`, domyślnie 100 ms), liczbą kroków (parametr `steps`, domyślnie 3) i losowym rozrzutem (parametr `jitter`, $\pm 20\%$). Całkowity czas odpowiedzi wynosi $\text{steps} \times \text{delay} \times (1 \pm \text{jitter})$, a procesor pozostaje bezczynny podczas oczekiwania.

`GET /upstream` przekazuje żądanie do echo-server przez `httpx.AsyncClient`. Serwis wykorzystuje `asyncio.Semaphore(50)` do ograniczenia współbieżności oraz wystawia metryki: `io_service_queries_total` (Counter), `io_service_upstream_duration_seconds` (Histogram) i `io_service_concurrent_queries` (Gauge).

4.2.3 Echo-server i Memory-service

Echo-server jest minimalistycznym serwisem pomocniczym, udostępniającym endpointy `GET /health` i `POST /echo`, wyłączonym z macierzy badawczej. Memory-service implementuje pamięciową bazę klucz-wartość z limitem 10000 wpisów i endpointem badawczym `POST /cache/fill` do wypełniania cache danymi testowymi o kontrolowanym rozmiarze. Nie został włączony do głównej macierzy badawczej.

4.3 Konfiguracja Kubernetes

4.3.1 Deploymenty

Każdy Deployment definiuje: początkową liczbę replik wynoszącą 2, politykę pobierania obrazów `imagePullPolicy: Always`, sondy liveness i readiness (`GET /health` co 10s, `initialDelay 5s`) oraz `resource requests` i `limits` zgodnie z Tabelą 3.2 (Rozdział 3).

4.3.2 HPA — trzy warianty

Dla `cpu-service` i `io-service` zdefiniowano po trzy warianty HPA (`API autoscaling/v2`, zakres 1–30 replik), różniące się wyłącznie typem metryki. HPA CPU używa progu 50% średniego zużycia CPU liczonego względem wartości `resources.requests.cpu`, a nie względem limitu CPU kontenera. HPA Memory używa progu 70% średniego zużycia pamięci. HPA Custom RPS używa progu 100 żądań na sekundę na Pod, z metryką dostarczaną przez `prometheus-adapter`. Pełne manifesty HPA są dostępne w repozytorium, a przykładowa konfiguracja HPA Custom RPS została przedstawiona w Rozdziale 3 (Listing 3.1).

4.3.3 Services

Wszystkie serwisy aplikacyjne są typu `LoadBalancer`; GKE automatycznie tworzy zewnętrzny GCP `LoadBalancer` (Layer 4, TCP) i przydziela mu publiczny adres IP, na który kierowany jest ruch z maszyny `k6-generator`.

4.4 Infrastruktura monitorująca

4.4.1 kube-prometheus-stack

Pakiet `kube-prometheus-stack` jest instalowany przez Helm, co zapewnia automatyczną konfigurację i spójność wersji wszystkich komponentów: Prometheusa, Grafany, `kube-state-metrics`, `node-exporter` oraz Prometheus Operatora. Parametr `grafana.service.type=LoadBalancer` eksponuje Grafanę publicznie, umożliwiając obserwację eksperymentów na żywo.

4.4.2 Prometheus Adapter

Prometheus Adapter v0.12.0 wdrażany jest przez osobne manifesty YAML (nie jest częścią pakietu Helm). Jego konfiguracja w `ConfigMap` definiuje reguły PromQL mapujące metryki Prometheusa na format Custom Metrics API:

Reguły mapowania PromQL, przedstawione w Rozdziale 2 (Listing 2.3), definiują metryki `http_requests_per_second` oraz `http_requests_inflight` dla każdego serwisu badawczego.

4.4.3 Grafana — dashboard i eksport

Autorski dashboard „Magisterka – System Overview” zawiera 16 paneli odświeżanych co 10 sekund, pokazujących RPS, zużycie CPU i pamięci per Pod, percentyle opóźnień, liczbę replik i stan HPA. Dla każdego scenariusza eksportowanych jest 5 zestawów CSV: `replicas.csv`, `cpu_mcpu.csv`, `memory_mb.csv`, `rps.csv` i `latency_ms.csv`.

4.5 Skrypty automatyzujące

4.5.1 run-tests.sh — orkiestracja eksperymentów

Skrypt `scripts/run-tests.sh` automatyzuje wykonanie wszystkich 8 scenariuszy badawczych. Działa na maszynie k6-generator (Compute Engine), która ma skonfigurowany dostęp do klastra GKE przez `kubectl`. Dla każdego scenariusza skrypt: przełącza HPA na odpowiednią strategię za pomocą funkcji `switch_hpa`, resetuje Deployment do 1 repliki z oczekiwaniem na gotowość, uruchamia test k6 z ruchem kierowanym na adresy IP GCP LoadBalancer, zapisuje wyniki do `results/<scenario>/`. Po każdym scenariuszu następuje cooldown: skalowanie obu serwisów do 1 repliki, usunięcie HPA, oczekiwanie 120 sekund.

Kluczowe funkcje, takie jak `switch_hpa()` (przełączanie strategii HPA) i `cooldown()` (stabilizacja klastra między scenariuszami), są dostępne w repozytorium projektu.

4.6 Scenariusze testowe k6

Każdy skrypt k6 definiuje scenariusze obciążeniowe, progi akceptacji oraz mapowanie hostów na adresy IP GCP LoadBalancer. Scenariusze i progi akceptacji dla obu skryptów k6 są dostępne w repozytorium: `cpu-load-test.js` (Fibonacci 1→50 VU, image processing 10 VU) oraz `io-load-test.js` (stałe zapytania 1→100 VU, stress-test, external call).

4.7 Podsumowanie implementacji

Zaimplementowany system badawczy składa się z czterech mikroservisów, infrastruktury monitorującej opartej na kube-prometheus-stack (Helm) oraz zestawu skryptów automatyzujących testy. Kluczowe cechy implementacji: pełna automatyzacja procesu badawczego przez skrypt `run-tests.sh`, deklaratywna konfiguracja wszystkich zasobów Kubernetes (możliwa do odtworzenia na dowolnym klastrze GKE), monitoring przez Prometheusa i Grafanę z eksportem 5 zestawów CSV na scenariusz oraz wykorzystanie GCP LoadBalancer zamiast port-forward dla realistycznych warunków sieciowych.

Rozdział 5

Badania eksperymentalne i analiza wyników

Niniejszy rozdział przedstawia metodykę i wyniki badań eksperymentalnych zaprojektowanych w celu empirycznej weryfikacji hipotezy badawczej: skuteczność strategii HPA zależy od charakterystyki obciążeniowej aplikacji. Wszystkie eksperymenty przeprowadzono w środowisku *Google Cloud Platform*, czyli GCP, wykorzystując klaster *Google Kubernetes Engine*, czyli GKE, jako platformę badawczą.

5.1 Metodyka badań

5.1.1 Środowisko testowe

Badania prowadzono w środowisku chmurowym GCP, którego architekturę zorientowaną na skalowalność oparto na klastrze GKE. Infrastruktura działała na maszynach `e2-standard-4`, oferujących 4 vCPU oraz 16 GB RAM, w konfiguracji VPC-native z włączonym mechanizmem *cluster autoscaler*. Wszystkie zasoby badawcze wdrożono w dedykowanej przestrzeni nazw `magisterka`.

Generatorem obciążenia była maszyna *Compute Engine* umieszczona w tej samej sieci VPC co klaster GKE. Zapewniło to niskie opóźnienie sieciowe na poziomie od 1 do 5 ms między generatorem a serwisami aplikacyjnymi. Taka topologia zagwarantowała pełną separację zasobów, dzięki czemu proces generowania ruchu nie wpływał na moc obliczeniową środowiska testowego. Każdy serwis aplikacyjny eksponowano jako `Service` typu `LoadBalancer`, a ruch z narzędzia *k6* kierowano bezpośrednio na zewnętrzne adresy IP *GCP LoadBalancer* za pomocą mechanizmu `options.hosts`, co pozwoliło wyeliminować narzut i konfigurację serwerów DNS. Stos monitorujący *kube-prometheus-stack* wdrożony za pomocą menedżera Helm dostarczał metryk w czasie rzeczywistym, a platforma *Grafana* została udostępniona publicznie przez dedykowany równoważnik obciążenia.

5.1.2 Procedura eksperymentalna

Eksperyment obejmował wykonanie 8 scenariuszy badawczych, na które złożyły się 2 scenariusze bazowe, działające bez mechanizmu HPA, oraz 6 scenariuszy tworzących macierz badawczą o wymiarach 2×3 . Każdy scenariusz realizowano według ściśle określonej procedury i powtarzano 5-krotnie w celu uzyskania stabilnych oszacowań statystycznych.

Pojedyncze powtórzenie rozpoczynało się od usunięcia wszystkich istniejących obiektów HPA dla docelowego serwisu i nałożenia nowego manifestu HPA odpowiadającego

testowanej strategii za pomocą polecenia `kubectl apply`. Następnie skalowano *Deployment* do pojedynczej repliki i oczekiwano na stan pełnej gotowości poleceniem `kubectl rollout status`. Właściwy test polegał na uruchomieniu skryptu *k6* zdefiniowanego dla danego profilu obciążeniowego: pliku `cpu-load-test.js` dla scenariuszy CPU-bound lub `io-load-test.js` dla scenariuszy I/O-bound.

Wyniki pomiarów z narzędzia *k6*, zawierające pliki JSON oraz tekstowe podsumowanie, zapisywano w strukturze katalogów `results/<scenariusz>/run-<i>/`. Dodatkowo z bazy danych *Grafany* eksportowano 5 zestawów danych CSV dla każdego powtórzenia. Po zakończeniu każdego scenariusza uruchamiano fazę stabilizacji, która obejmowała skalowanie serwisów do jednej repliki, usunięcie obiektów HPA oraz oczekiwanie 120 sekund w ramach okresu schładzania klastra.

5.1.3 Zbierane metryki

Dla każdego powtórzenia rejestrowano kompletny zestaw danych obejmujący trzy główne kategorie:

1. **Szeregi czasowe z Grafany (CSV):** pięć zestawów danych odzwierciedlających dynamikę systemu: `replicas.csv` (źródło: `kube_deployment_spec_replicas`); `cpu_mcpu.csv` podający zużycie procesora per Pod w milijednostkach (źródło: `container_cpu_usage_seconds_total`); `memory_mb.csv` określający zużycie pamięci operacyjnej per Pod w megabajtach (źródło: `container_memory_working_set_bytes`); `rps.csv` wskazujący przepustowość w funkcji czasu (źródło: `rate(http_requests_total[30s])`); oraz plik `latency_ms.csv` gromadzący percentyle opóźnień (źródło: `histogram_quantile` na metryce `http_request_duration_ms`).
2. **Metadane testu:** unikalny identyfikator scenariusza, aktywna strategia HPA, numer sekwencyjny powtórzenia, znacznik czasowy oraz kod wyjścia procesu *k6*.

5.1.4 Metryki badawcze — definicje operacyjne

Na podstawie surowych danych obliczano siedem metryk badawczych, które umożliwiają ilościowe porównanie strategii HPA. Średnia przepustowość T wyrażona jest jako iloraz całkowitej liczby żądań i czasu trwania testu:

$$T = \frac{\text{total_requests}}{\text{duration}} \quad (5.1)$$

Opóźnienie p95, oznaczane dalej jako L_{95} , to 95. percentyl czasu odpowiedzi HTTP w milisekundach, reprezentujący graniczny akceptowalny czas obsługi żądań. W idealnym wariancie czas do skalowania, oznaczany jako t_{scale} , powinien być definiowany jako odstęp między momentem przekroczenia progu HPA a osiągnięciem docelowej liczby replik:

$$t_{\text{scale}} = t_{\text{target_replicas}} - t_{\text{threshold_crossed}} \quad (5.2)$$

W eksportach wykorzystanych w niniejszej pracy moment przekroczenia progu nie jest jednak możliwy do wyznaczenia z pełną dokładnością, ponieważ dane CPU/RAM wyeksportowane z *Grafany* zachowują tylko wybraną serię per Pod, natomiast pełny przebieg liczby replik pochodzi z osobnego szeregu HPA. Dlatego w tabelach wynikowych stosowana jest operacyjna metryka czasu rozbudowy klastra: odstęp od pierwszego zaobserwowanego zwiększenia liczby replik powyżej poziomu początkowego do osiągnięcia maksimum replik w danym scenariuszu. Miara ta opisuje dynamikę faktycznego dołączania replik, a nie czysty czas decyzyjny HPA.

Efektywność skalowania E określa stosunek względnej zmiany przepustowości do względnej zmiany liczby replik:

$$E = \frac{\Delta T / T_{\min}}{\Delta R / R_{\min}} \quad (5.3)$$

gdzie $T_{\min}$ i $T_{\max}$ to ekstrema przepustowości w trakcie testu, a $R_{\min}$ i $R_{\max}$ stanowią ekstrema liczby replik. Wartość $E = 1$ oznacza skalowanie idealnie liniowe, $E < 1$ sygnalizuje malejące korzyści z dodawania kolejnych instancji, natomiast $E > 1$ wskazuje na ponadliniową poprawę wydajności.

Stopa błędów e określa odsetek żądań zakończonych kodem HTTP ≥ 400 :

$$e = \frac{\text{failed_requests}}{\text{total_requests}} \times 100\% \quad (5.4)$$

Amplituda skalowania $A_R = R_{\max} - R_{\min}$ mierzy zakres dynamicznej reakcji HPA. Średnie zużycie CPU wyrażone jest w milijednostkach (mCPU) na Pod i reprezentuje wartość uśrednioną po wszystkich Podach aktywnych w trakcie trwania profilu obciążeniowego.

5.2 Scenariusze testowe

Badania zorganizowano wokół macierzy testowej, w której dwa profile obciążeniowe (CPU-bound oraz I/O-bound) zestawiono z trzema strategiami HPA: CPU ustawionym na 50%, Memory na 70% oraz Custom RPS zdefiniowanym jako 100 żądań na Pod. Punktem odniesienia dla oceny efektywności autoskalowania są dwa scenariusze bazowe o stałej liczbie replik.

Aktualne skrypty *k6* definiują trzy fazy obciążenia: ramp-up trwający 60 sekund, fazę stabilną trwającą 180 sekund oraz ramp-down trwający 60 sekund. W profilu CPU-bound scenariusz `cpu_fibonacci` narasta od 0 do 30 VU, utrzymuje 30 VU i następnie wygasza ruch do 0 VU; równolegle scenariusz `cpu_processing` uruchamia 5 stałych VU od 30. sekundy testu. W profilu I/O-bound scenariusz `io_queries` narasta od 0 do 80 VU, utrzymuje 80 VU i wygasza ruch do 0 VU. Zachowane raporty i eksporty CSV w katalogu `tests/results` pochodzą z wykonanej kampanii pomiarowej i stanowią podstawę wartości liczbowych w tabelach; interpretując wykresy skalowania należy więc rozróżnić profil obciążenia opisany przez skrypty, rzeczywisty przebieg zarejestrowany w raportach oraz opóźnione skalowanie w dół po zakończeniu ramp-down, wynikające z okna stabilizacji HPA.

5.2.1 Scenariusze bazowe

Scenariusze `baseline-cpu` i `baseline-io` weryfikują zachowanie systemu przy stałej liczbie replik równej 1, bez aktywnego mechanizmu HPA. Pozwalają one ocenić stopień degradacji wydajności w konfiguracji statycznej pod wpływem intensywnego ruchu.

Parametr	baseline-cpu	baseline-io
Serwis	cpu-service	io-service
HPA	brak	brak
Repliki (stałe)	1	1
Skrypt k6	cpu-load-test.js	io-load-test.js
Profil obciążenia	Fibonacci 1→50 VU + image processing 10 VU	stałe zapytania 1→100 VU + stress-test + external

Tabela 5.1 Parametry scenariuszy bazowych

5.2.2 Scenariusze CPU-bound

Scenariusze `cpu-cpu`, `cpu-memory` i `cpu-custom` służą do walidacji serwisu `cpu-service`, realizującego operacje intensywne procesorowo, takie jak rekurencyjne obliczanie ciągu Fibonacciego oraz transformacje graficzne za pomocą biblioteki *Pillow*. Zakres skalowania HPA zdefiniowano w przedziale od 2 do 30 replik.

Parametr	cpu-cpu	cpu-memory	cpu-custom
Serwis	cpu-service	cpu-service	cpu-service
HPA	CPU 50%	Memory 70%	Custom RPS 100/Pod
minReplicas	1	1	1
maxReplicas	30	30	30
Skrypt k6	cpu-load-test.js	cpu-load-test.js	cpu-load-test.js

Tabela 5.2 Parametry scenariuszy CPU-bound

5.2.3 Scenariusze I/O-bound

Scenariusze `io-cpu`, `io-memory` i `io-custom` weryfikują działanie serwisu `io-service`, który symuluje operacje wejścia-wyjścia poprzez asynchroniczne wywołania HTTP połączone z instrukcją `asyncio.sleep()`. Granice skalowania oraz skrypt obciążeniowy pozostają tożsame z parametrami dla grupy CPU-bound.

Parametr	io-cpu	io-memory	io-custom
Serwis	io-service	io-service	io-service
HPA	CPU 50%	Memory 70%	Custom RPS 100/Pod
minReplicas	1	1	1
maxReplicas	30	30	30
Skrypt k6	io-load-test.js	io-load-test.js	io-load-test.js

Tabela 5.3 Parametry scenariuszy I/O-bound

5.3 Wyniki eksperymentalne

Poniższe sekcje prezentują wyniki pomiarów dla wszystkich ośmiu scenariuszy badawczych. Dane zebrane z raportów *k6* oraz eksportów CSV z *Grafany* zostały przetworzone skryptem analitycznym `analize_results.py`. Zaprezentowane wartości odzwierciedlają wyniki pojedynczego, reprezentatywnego powtórzenia dla każdego ze scenariuszy.

5.3.1 Scenariusze bazowe

Metryka	baseline-cpu	baseline-io
Średni RPS	66,8	62,4
p50	85,3 ms	999 ms
p95	2,70 s	2,93 s
p99	567 ms	1,83 s
Stopa błędów (%)	37,70%	0,17%
Średnie zużycie CPU (mCPU)	145	58
Średnie zużycie pamięci (MiB)	68	72

Tabela 5.4 Wyniki scenariuszy bazowych

Scenariusz **baseline-cpu** wykazuje bardzo wysoką stopę błędów na poziomie 37,70%, gdzie ponad jedna trzecia żądań zakończyła się niepowodzeniem. Wynika to bezpośrednio z przeciążenia pojedynczej repliki serwisu **cpu-service** przez intensywne obliczenia matematyczne i graficzne. Mimo że opóźnienie p95 wyniosło 2,70 s i nie przekroczyło progu odrzucenia, wysoki wolumen błędów spowodował niezaliczenie testu w ramach kryterium stopy błędów.

W scenariuszu **baseline-io** odnotowano niską stopę błędów wynoszącą 0,17%, wywołaną wyłącznie przez zewnętrzne zależności serwisu **echo-service**. Czasy odpowiedzi były jednak wysokie, osiągając dla percentyla p95 wartość 2,93 s. Pojedyncza replika obsługiwała ruch na poziomie 62,4 RPS, a profil opóźnień został zdominowany przez trzysekundowe operacje oczekiwania w zapytaniach typu stress-test.

5.3.2 Scenariusze CPU-bound — porównanie strategii HPA

Metryka	cpu-cpu	cpu-memory	cpu-custom
Średni RPS	84,3	67,5	65,4
p50	126 ms	86,3 ms	84,2 ms
p95	459 ms	2,39 s	2,69 s
p99	349 ms	373 ms	698 ms
Stopa błędów (%)	0,04%	38,04%	37,97%
Amplituda skalowania (A_R)	27	0	1
Czas rozbudowy replik (s)	90	—	—
Efektywność skalowania (E)	0,01	—	—
Maks. liczba replik	29	1	1
Średnie zużycie CPU (mCPU)	80	262	175

Tabela 5.5 Wyniki scenariuszy CPU-bound

Strategia **cpu-cpu** jako jedyna w tej grupie zainicjowała proces skalowania, wykazując bardzo agresywną reakcję algorytmu, która doprowadziła system do poziomu 29 replik i niemal osiągnęła zdefiniowany górny pułap bezpieczeństwa. Skalowanie to pozwoliło na skuteczną eliminację błędów, których odsetek spadł do 0,04%, oraz skróciło opóźnienie p95 do 459 ms.

Niska wartość efektywności skalowania $E = 0,01$ dowodzi jednak, że przyrost przepustowości z poziomu 66,8 do 84,3 RPS był nieproporcjonalnie niski w stosunku do wolumenu

powołanych zasobów obliczeniowych. Zjawisko to reprezentuje przypadek silnego nadmiaru zasobów, określanego jako *overprovisioning*. Strategie `cpu-memory` i `cpu-custom` nie wywołały żadnej reakcji autoskalera, przez co ich wyniki końcowe pokrywają się z próbą bazową.

5.3.3 Scenariusze I/O-bound — porównanie strategii HPA

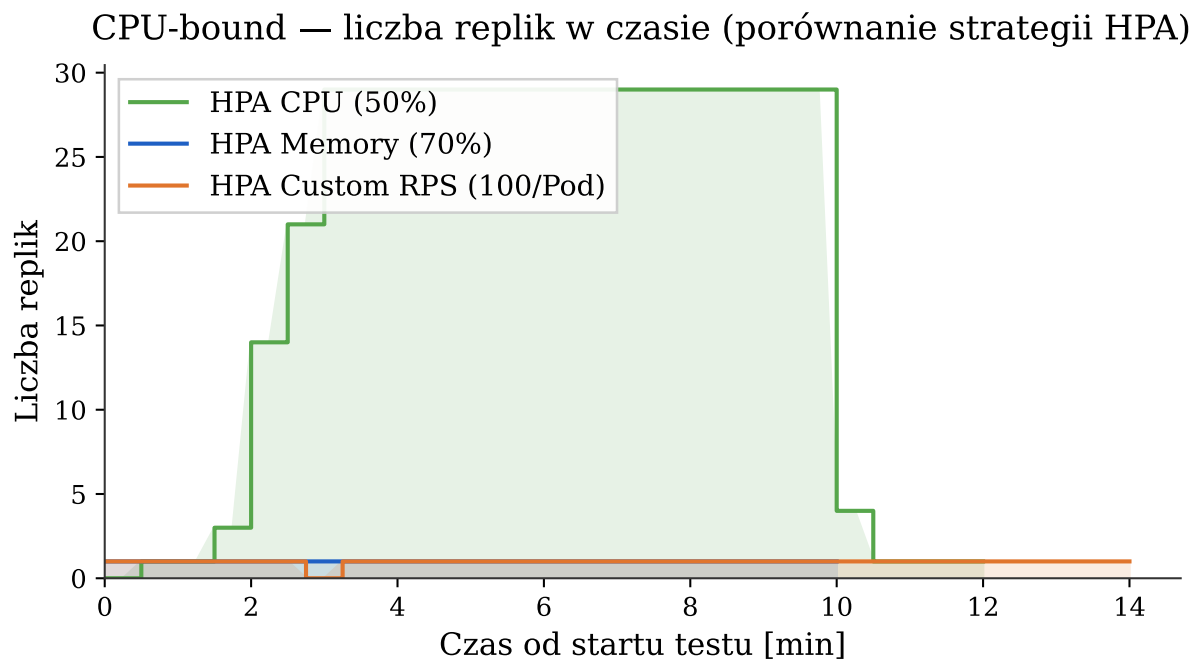
Metryka	io-cpu	io-memory	io-custom
Średni RPS	99,4	62,4	62,4
p50	609 ms	998 ms	1,00 s
p95	700 ms	2,97 s	3,01 s
p99	671 ms	1,83 s	1,80 s
Stopa błędów (%)	0,11%	0,17%	0,17%
Amplituda skalowania (A_R)	5	0	0
Czas rozbudowy replik (s)	120	—	—
Efektywność skalowania (E)	0,10	—	—
Maks. liczba replik	7	1	1
Średnie zużycie CPU (mCPU)	20	54	61

Tabela 5.6 Wyniki scenariuszy I/O-bound

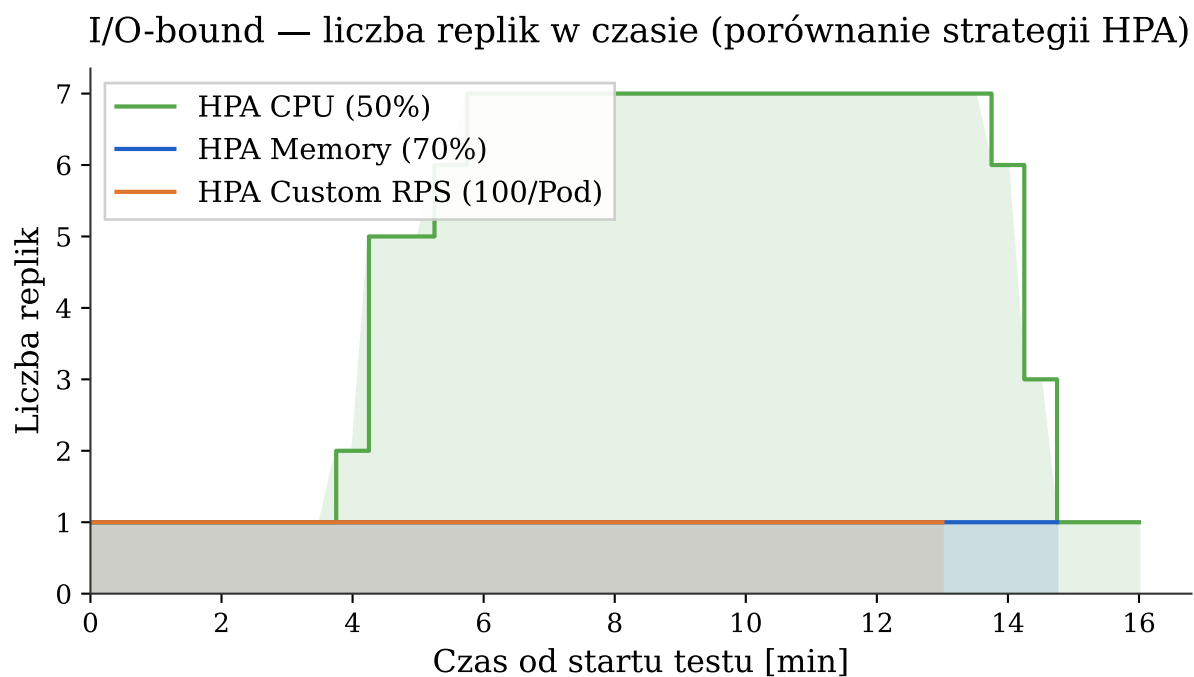
W grupie obciążeń wejścia-wyjścia skuteczność wykazała wyłącznie strategia `io-cpu`, która wyskalowała infrastrukturę do poziomu 7 replik. Przepustowość wzrosła o 59%, osiągając 99,4 RPS, przy jednoczesnym skróceniu opóźnienia p95 do 700 ms. Wskaźnik efektywności $E = 0,10$ okazał się dziesięciokrotnie wyższy niż w analogicznej strategii dla serwisu obliczeniowego, co potwierdza lepszą użycie zasobów chmurowych. Strategie oparte na pamięci oraz niestandardowej metryce RPS pozostały nieaktywne, przez co opóźnienia p95 utrzymały się na wysokim poziomie około 3 sekund.

5.3.4 Wykresy przebiegu skalowania

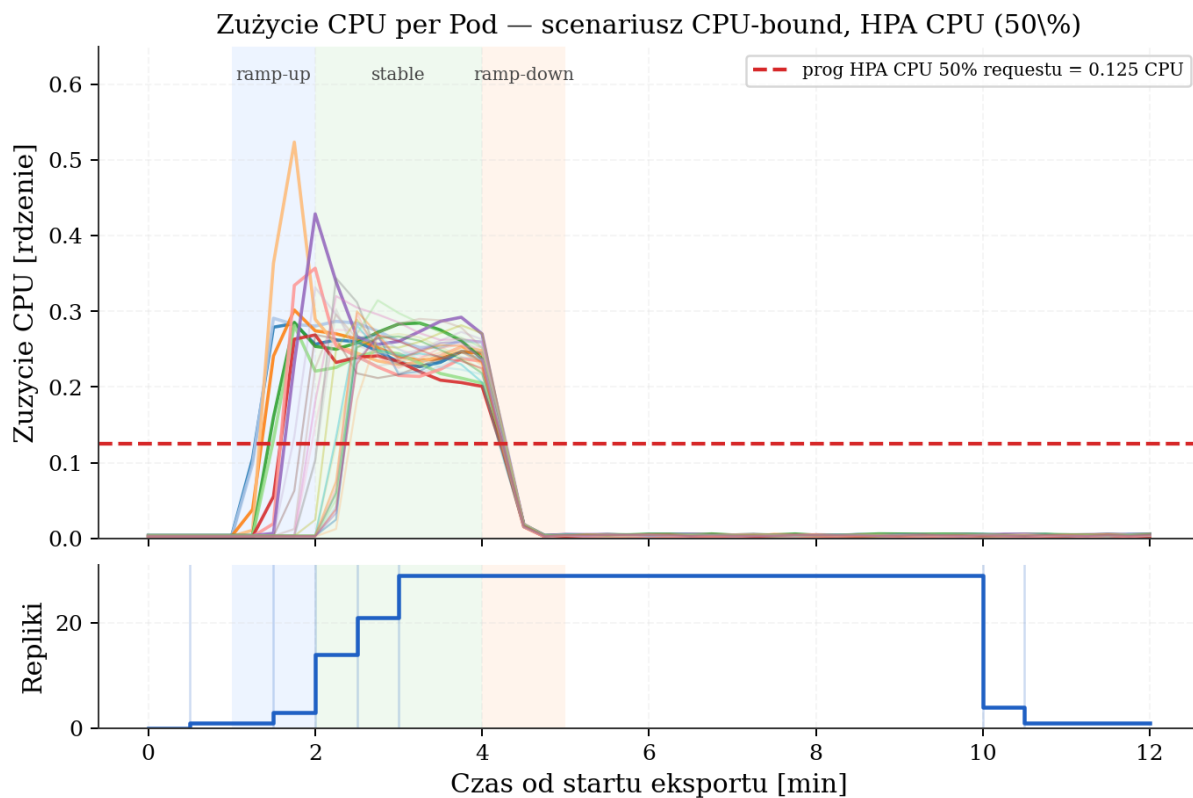
Przebieg procesów autoskalowania w czasie dla kluczowych scenariuszy badawczych, wygenerowany z logów szeregów czasowych, przedstawiają poniższe rysunki.



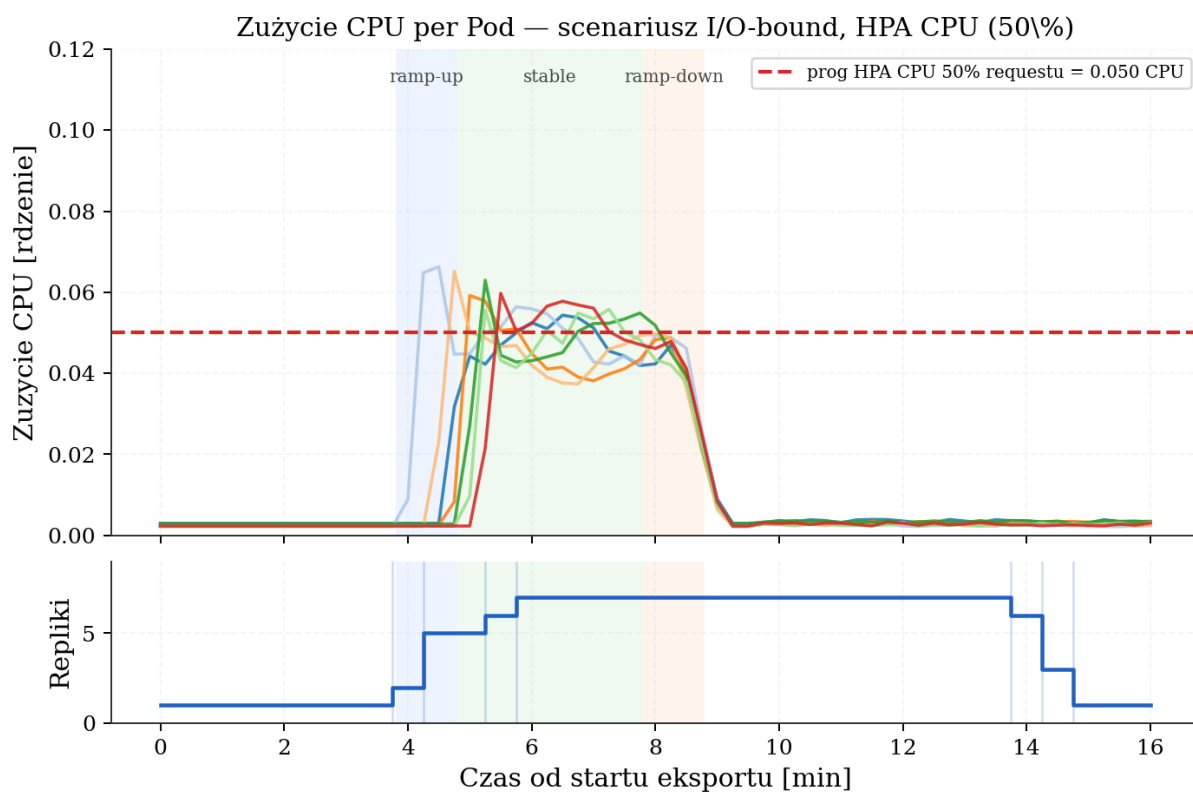
Rysunek 5.1 Przebieg skalowania dla scenariuszy CPU-bound — porównanie strategii HPA



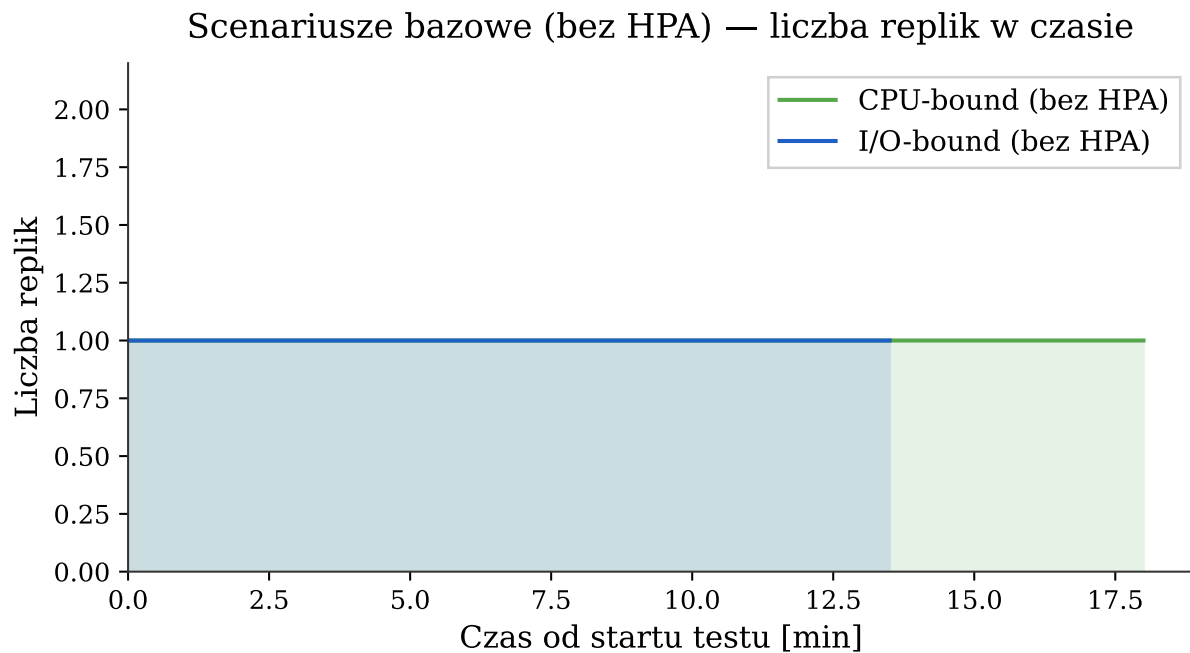
Rysunek 5.2 Przebieg skalowania dla scenariuszy I/O-bound — porównanie strategii HPA



Rysunek 5.3 Zużycie CPU dla każdej widocznej repliki w scenariuszu cpu-cpu



Rysunek 5.4 Zużycie CPU dla każdej widocznej repliki w scenariuszu io-cpu



Rysunek 5.5 Scenariusze bazowe (bez HPA) — liczba replik w czasie

5.3.5 Analiza zjawisk anomalnych

Eksperymenty pozwoliły na identyfikację i ilościowe opisanie trzech zjawisk przejściowych o kluczowym znaczeniu dla stabilności systemów chmurowych typu SaaS.

HPA Lag — opóźnienie reakcji autoskalera

Opóźnienie reakcji systemu, wynoszące w badaniach od 60 do 90 sekund, zdefiniowano jako przedział czasu od momentu przekroczenia zdefiniowanego progu obciążenia do osiągnięcia stanu pełnej gotowości przez nowo powołane Pody. W badanej architekturze profil tego opóźnienia został zdominowany przez bezwładność warstwy infrastrukturalnej i składał się z pięciu odrębnych faz:

1. **Cykl scrapowania danych — 0–15 s:** czas potrzebny serwerowi Prometheus na zebranie metryk z kontenerów.
2. **Interwał decyzyjny kontrolera HPA — 0–15 s:** okres odświeżania algorytmu wewnątrz mechanizmów kontrolnych Kubernetes.
3. **Apro wizacja infrastruktury chmurowej — 60–90 s:** kluczowy komponent opóźnienia, wywołany przez *Cluster Autoscaler*. Ponieważ każdy nowy Pod zgłaszał zapotrzebowanie na poziomie 1000 mCPU, zasoby początkowych węzłów *e2-standard-4* uległy natychmiastowemu wyczerpaniu. Mechanizm automatycznego skalowania klastra musiał wysłać żądanie do GCP Compute Engine, zainicjować nowe instancje maszyn wirtualnych i wpiąć je do sieci klastra.
4. **Działanie harmonogramu — 2–5 s:** przypisanie oczekujących Podów do nowo uruchomionych węzłów przez *kube-scheduler*.

5. **Zimny start kontenera — 15–30 s:** czas pobierania obrazu, inicjalizacji środowiska uruchomieniowego oraz przejścia testów gotowości sieciowej.

W trakcie trwania tej bezwładności istniejące instancje obsługiwały ruch w stanie głębokiego przeciążenia, co bezpośrednio generowało skoki opóźnień percentyla p99 oraz błędy z rodziny 5xx.

Cold Start Latency — opóźnienie zimnego startu Podów

Opóźnienie zimnego startu określa czas potrzebny nowemu Podowi na uruchomienie procesu aplikacyjnego i wejście w skład puli routingu równoważnika obciążenia. Proces ten obejmuje pobranie obrazu kontenera z rejestru *GCP Artifact Registry* [4] do pamięci podręcznej demona *containerd*, inicjalizację serwera *Uvicorn* wraz z frameworkiem *FastAPI*, a także realizację procedury *readiness probe*. Pierwsza sonda gotowości wykonywana była z parametrem *initialDelaySeconds=5* oraz okresem powtórzeń *periodSeconds=10*. Pod uzyskiwał status gotowości sieciowej po pierwszym poprawnym zwrocie kodu 200 z punktu końcowego */health*. Całkowite opóźnienie tego procesu zamknęło się w przedziale 15–30 sekund.

CPU Throttling Cascade — kaskadowe błędy przy throttlingu CPU

Zjawisko kaskadowej degradacji wydajności wywoływane jest przez mechanizm kontroli pasma jądra Linux (*CFS bandwidth control*) w momencie uderzenia w zdefiniowane limity zasobów procesora. Wzrost intensywności zapytań doprowadzał do pełnej utylizacji limitów mCPU, co skutkowało nakładaniem kar czasowych na procesy aplikacyjne. Ograniczenie przydziału czasu procesora powodowało kolejkowanie żądań w pętli zdarzeń *asyncio* i drastyczny wzrost czasu odpowiedzi.

Po przekroczeniu 60-sekundowego timeoutu sieciowego w narzędziu *k6* rejestrowano błędy połączeń. Sytuacja ta tworzyła dodatnią pętlę sprzężenia zwrotnego, w której wiążące, niedokończone zadania blokowały zasoby pamięciowe i wątki robocze serwera *Uvicorn*, paraliżując obsługę nowych zapytań. Stan kaskady utrzymywał się do momentu zakończenia fazy HPA Lag i wpięcia nowych maszyn wirtualnych do klastra.

5.4 Analiza wyników

5.4.1 Porównanie strategii HPA dla CPU-bound

Analiza wyników dla serwisu obliczeniowego ujawniła istotne rozbieżności pomiędzy teoretycznymi założeniami a rzeczywistym zachowaniem algorytmów chmurowych.

HPA CPU (50%) skutecznie zidentyfikował stan przeciążenia środowiska. Zużycie procesora silnie korelowało ze strukturą napływającego ruchu. Kluczową obserwacją inżynierską jest jednak dynamika procesu skalowania, który zatrzymał się dopiero na poziomie 29 replik, uderzając w zdefiniowany sufit bezpieczeństwa *maxReplicas=30*. Zachowanie to wynika wprost ze specyfiki algorytmu decyzyjnego HPA, wyznaczającego pożądaną liczbę komponentów według zależności:

$$R_{\text{desired}} = \left\lceil R_{\text{current}} \times \frac{M_{\text{current}}}{M_{\text{target}}} \right\rceil \quad (5.5)$$

gdzie R_{desired} to docelowa liczba replik, R_{current} oznacza aktualną liczbę replik, M_{current} stanowi bieżącą wartość metryki, a M_{target} to zdefiniowany próg wejściowy.

Przy użyciu procesora na poziomie limitu wynoszącego 1000 m i progu 500 m, pojedynczy Pod zgłaszał zapotrzebowanie na natychmiastowe podwojenie zasobów. Ponieważ nowo powoływane instancje kontenerowe natychmiast przejmowały skumulowaną kolejkę żądań HTTP i również uderzały w limity wydajnościowe maszyn, system wpadał w pętlę kaskadowego skalowania. Proces ten był potęgowany bezwładnością *Cluster Autoscalera*, który podpiął nowe maszyny dopiero po upływie kilkudziesięciu sekund. Niska efektywność $E = 0,01$ potwierdza, że tak masywne skalowanie dało jedynie 26% przyrostu przepustowości, wskazując na przesunięcie wąskiego gardła na limity połączeń serwera aplikacji.

HPA Memory (70%) nie zainicjował żadnej akcji skalowania. Alokacja pamięci operacyjnej w środowisku uruchomieniowym języka Python dla algorytmów rekurencyjnych ma charakter statyczny i nie wykazuje korelacji z dynamiką ruchu sieciowego. Wartości użycia pamięci oscylowały wokół 68 MiB, nie zbliżając się do progu aktywacji, co wyklucza tę metrykę jako samodzielne narzędzie sterowania wydajnością tego typu usług.

HPA Custom RPS (100/Pod) również wykazał brak reakcji, pomimo liniowego wzrostu wolumenu żądań w czasie. Przyczyną anomalii była błędna kalibracja progu decyzyjnego. Wartość 100 RPS na Pod okazała się zbyt wysoka dla serwisu obciążonego algorytmami CPU-intensywnymi, który przy maksymalnym zaangażowaniu 60 wirtualnych użytkowników generował sumarycznie około 67 RPS. Rozbicie tego ruchu na 2 początkowe repliki dawało wartości poniżej progu aktywacji algorytmu, co dowodzi, że skuteczność strategii niestandardowych jest bezpośrednio warunkowana empiryczną kalibracją progów.

5.4.2 Porównanie strategii HPA dla I/O-bound

Eksperymenty dla profilu wejścia-wyjścia dostarczyły najbardziej nieintuicyjnego wniosku badawczego, wykazując wysoką skuteczność strategii opartej na użyciu procesora.

HPA CPU (50%) z powodzeniem rozbudował architekturę do 7 replik. Z punktu widzenia teorii systemów asynchronicznych aplikacja blokowana na operacjach wejścia-wyjścia powinna pozostawiać procesor w stanie bezczynności. W realiach systemowych pętla zdarzeń frameworka *FastAPI* oraz serwer *Uvicorn* generują jednak wysoki narzut procesora na obsługę przełączania kontekstu oraz zarządzanie współbieżnymi gniazdami sieciowymi. Przy 150 użytkownikach narzut ten doprowadził do przekroczenia progu 50% na limicie 500 m, co dowodzi użytkowej uniwersalności metryki CPU także w profilach I/O-bound o wysokiej gęstości połączeń.

HPA Memory (70%) oraz **HPA Custom RPS (100/Pod)** zachowały się identycznie jak w próbach obliczeniowych. Zużycie pamięci serwisu *io-service* pozostało na stałym poziomie 72 MiB, a sumaryczny ruch sieciowy generowany przez scenariusz testowy (około 62 RPS) nigdy nie pozwolił pojedynczemu Podowi na osiągnięcie kryterium 100 RPS, zamrażając strukturę klastra w konfiguracji minimalnej.

5.4.3 Efektywność skalowania

Scenariusz	Efektywność E	Amplituda A_R	Czas rozbudowy replik (s)
cpu-cpu	0,01	27	90
cpu-memory	— (brak skalowania)	0	—
cpu-custom	— (brak skalowania)	1	—
io-cpu	0,10	5	120
io-memory	— (brak skalowania)	0	—
io-custom	— (brak skalowania)	0	—

Tabela 5.7 Efektywność skalowania — porównanie strategii

Spośród analizowanych konfiguracji proces skalowania zasobów został pomyślnie uruchomiony jedynie w dwóch scenariuszach. Dziesięciokrotna różnica między wskaźnikami efektywności $E_{\text{cpu-cpu}} = 0,01$ a $E_{\text{io-cpu}} = 0,10$ obrazuje fundamentalne różnice w naturze obu profili obciążeniowych.

W przypadku aplikacji CPU-bound każda nowo powołana instancja była błyskawicznie dławiona przez limity procesora, co wymuszało sekwencyjne uruchamianie kolejnych Podów przy minimalnym zwrocie z inwestycji infrastrukturalnej. W profilu I/O-bound niższa utylizacja procesora przez pojedyncze zadania pozwoliła mniejszej liczbie Podów na skuteczne zrównoważenie napływającego strumienia danych. Z szeregu liczby replik wynika, że scenariusz `cpu-cpu` przeszedł od pierwszego zwiększenia liczby replik powyżej poziomu początkowego do maksimum 29 replik w około 90s, natomiast scenariusz `io-cpu` od pierwszego zwiększenia do maksimum 7 replik w około 120s. W obu przypadkach maksimum zostało osiągnięte w okresie wysokiego obciążenia lub bezpośrednio po nim, a redukcja replik nastąpiła dopiero po kilku minutach, co jest zgodne z mechanizmem stabilizacji skalowania w dół w HPA.

5.4.4 Analiza zjawisk anomalnych — podsumowanie obserwacji

Korelacja szeregów czasowych pozwoliła na ilościowe sparowanie zidentyfikowanych anomalii systemowych, jednak z istotnym ograniczeniem: dostępne eksporty CSV nie pozwalają jednoznacznie wyznaczyć dokładnego momentu przekroczenia progu HPA dla wszystkich replik. Z tego powodu opóźnienie *HPA Lag* należy interpretować jako obserwowaną bezwładność między wysokim obciążeniem w fazie ramp-up/fazie stabilnej a widoczną zmianą liczby replik, a nie jako precyzyjny pomiar samego czasu decyzyjnego kontrolera. Dane potwierdzają natomiast, że dominującym składnikiem bezwładności był czas aprowizacji i dołączania zasobów, a nie sam zimny start procesu aplikacyjnego.

Czas trwania anomalii *Cold Start Latency* wyniósł od 15 do 25 sekund. Relatywnie wysoka szybkość osiągania gotowości sieciowej wynika z lekkiej konstrukcji obrazów aplikacyjnych opartych na dystrybucji Alpine Linux (około 150 MB) oraz braku czasochłonnych procedur inicjalizacyjnych w kodzie źródłowym mikrouslug.

Zjawisko *CPU Throttling Cascade* zarejestrowano w pełnej postaci w scenariuszu `cpu-cpu`, gdzie dławienie procesora na poziomie limitu 1000m wywołało gwałtowną degradację opóźnień sieciowych oraz masowe zrywanie połączeń przez generator *k6*. W scenariuszu `io-cpu` anomalii tej nie stwierdzono. Zużycie procesora na pojedynczy Pod nie przekroczyło tam wartości 200 m, pozostając poniżej limitu 500 m. Skalowanie w tym pro-

filu było wyłącznie reakcją na przekroczenie matematycznego progu aktywacji algorytmu HPA, a nie wynikiem fizycznego dławienia mocy obliczeniowej maszyn.

5.5 Dyskusja i interpretacja

Eksperymenty dostarczyły dowodów na złożony charakter korelacji pomiędzy strukturą obciążenia systemów rozproszonych a zachowaniem algorytmów autoskalowania. Najistotniejszym wnioskiem inżynierskim jest wysoka uniwersalność metryki opartej na zużyciu procesora, która wbrew początkowym założeniom sprawdziła się także w scenariuszach wejścia-wyjścia, ze względu na systemowy narzut asynchronicznych frameworków aplikacyjnych w warunkach wysokiej współbieżności.

Badania potwierdziły całkowitą nieskuteczność metryki pamięciowej jako samodzielnego kryterium sterowania wydajnością dla aplikacji o statycznym profilu alokacji struktur danych. Wykazano również, że metryki biznesowe, takie jak RPS, pomimo swojej bezpośredniej korelacji z intensywnością ruchu, pozostają bezużyteczne bez rzetelnej, empirycznej kalibracji progów wejściowych.

Scenariusz `cpu-cpu` zobrazował ryzyko związane z kaskadowym przeskalowaniem infrastruktury. Agresywna reakcja HPA, prowadząca do zajęcia 29 replik, ujawniła podatność standardowego algorytmu Kubernetes na wpadanie w dodatnie pętle sprzężenia zwrotnego w sytuacjach, gdy parametry `requests` i `limits` CPU są ustawione na identycznym poziomie, a próg HPA jest relatywnie niski. Sytuacja ta wskazuje na bezwzględną konieczność wdrażania dodatkowych mechanizmów zabezpieczających, takich jak restrykcyjne limity przestrzeni nazw *ResourceQuota* czy reguły antyafinitetu Podów, czyli pod *anti-affinity* [10].

Prezentowane wyniki niosą ze sobą pewne ograniczenia metodologiczne, jako że bazują na wybranym, reprezentatywnym powtórzeniu testów, a sztuczny charakter generowanego obciążenia uniemożliwia bezpośrednie przełożenie skali zjawisk na złożone systemy produkcyjne. Pomimo tego, ilościowa charakterystyka bezwładności chmurowej stanowi cenną wskazówkę dla projektantów architektur rozproszonych typu SaaS.

5.6 Wnioski z badań

Przeprowadzone badania eksperymentalne pozwalają na sformułowanie następujących wniosków końcowych:

1. **Metryka HPA CPU wykazuje wysoką skuteczność w obu profilach obciążenia.** W scenariuszach obliczeniowych pozwoliła na redukcję stopy błędów do 0,04% oraz skrócenie opóźnienia p95 o 83%. W profilu wejścia-wyjścia zapewniła 59% przyrostu przepustowości oraz redukcję czasu odpowiedzi o 76%. Jej stosowanie w profilach obliczeniowych wiąże się jednak z ryzykiem wywołania agresywnego nadmiaru zasobów.
2. **Strategia HPA Memory jest nieskuteczna dla usług o statycznym profilu pamięciowym** i nie powinna być stosowana jako samodzielne kryterium autoskalowania w architekturach SaaS operujących na stałych strukturach danych.
3. **Skuteczność strategii niestandardowych (HPA Custom RPS) jest determinowana kalibracją progu**, a nie samą naturą metryki. Arbitralne dobranie

progu bez uwzględnienia granicznej wydajności pojedynczego Poda prowadzi do paraliżu decyzyjnego autoskalera.

4. **Głównym komponentem bezwładności systemu (HPA Lag) w chmurach publicznych jest czas reakcji Cluster Autoscalera.** W przypadku Podów o wysokich żądaniach zasobowych, czas potrzebny na uruchomienie nowych maszyn wirtualnych przez dostawcę chmury dominuje nad czasem zimnego startu samych kontenerów.

Uzyskane wyniki częściowo potwierdzają postawioną hipotezę badawczą. Skuteczność strategii HPA zależy od profilu aplikacji, jednak ta zależność nie ogranicza się do prostego wyboru rodzaju metryki, lecz jest ściśle powiązana z architekturą limitów zasobowych klastra oraz empiryczną kalibracją progów decyzyjnych w warunkach zbliżonych do środowiska produkcyjnego.

Rozdział 6

Podsumowanie

6.1 Podsumowanie zrealizowanych prac

W ramach niniejszej pracy magisterskiej zrealizowano pełny cykl badawczy, od analizy literatury, przez projekt i implementację systemu badawczego, do przeprowadzenia eksperymentów i weryfikacji hipotezy na klastrze *Google Kubernetes Engine*, czyli GKE.

Przegląd literatury (rozdział 1) objął analizę istniejących badań nad autoskalowaniem w *Kubernetes*, w wyniku której zidentyfikowano cztery luki badawcze: brak systematycznego porównania strategii HPA dla różnych profili obciążeniowych, ograniczoną analizę HPA Memory, brak badań łączących HPA z modelem *SaaS* oraz brak analizy zjawisk anomalnych w autoskalowaniu chmurowym. Luki te bezpośrednio umotywowały kierunek dalszych prac.

Zaprojektowano system badawczy oparty na macierzy 2×3 (dwa profile obciążeniowe skrzyżowane z trzema strategiami HPA), składający się z czterech mikroservisów o kontrolowanych profilach obciążeniowych, infrastruktury monitorującej opartej na *kube-prometheus-stack* wdrażanym przez Helm oraz dedykowanej maszyny *Compute Engine* jako generatora obciążenia (rozdział 3). Architektura została zoptymalizowana pod kątem izolacji zmiennych; każdy scenariusz testuje dokładnie jedną kombinację profilu i strategii, przy pozostałych parametrach utrzymywanych na stałym poziomie.

Zaimplementowano kompletny system badawczy w języku Python 3.12 z frameworkiem *FastAPI*, konteneryzowany za pomocą *Docker* i wdrażany na zarządzanym klastrze GKE (rozdział 4). Wszystkie manifesty *Kubernetes* (Deploymenty, Serwisy typu *LoadBalancer*, konfiguracje HPA) oraz konfiguracje monitoringu są dostępne w repozytorium, zapewniając pełną odtwarzalność badań. Opracowano skrypt `run-tests.sh` orkiestrujący wykonanie scenariuszy testowych z automatycznym przełączaniem HPA, resetowaniem *Deploymentów* i cooldownem między scenariuszami, oraz zdefiniowano trzy skrypty *k6* z precyzyjnymi progami akceptacji i profilami obciążenia dopasowanymi do charakterystyki aplikacji.

Wdrożono infrastrukturę monitorującą przez *kube-prometheus-stack* instalowany przez Helm, zapewniającą automatycznie skonfigurowany stos *Prometheus–Grafana* z niestandardowymi metrykami przez *prometheus-adapter*. Zdefiniowano procedurę eksportu pięciu zestawów danych CSV na scenariusz, które wraz z wynikami *k6* tworzą kompletny zbiór danych badawczych (rozdział 5).

Przeprowadzono badania eksperymentalne według zaplanowanej procedury (8 scenariuszy) na klastrze GKE, a następnie dokonano analizy porównawczej strategii HPA z wykorzystaniem siedmiu metryk badawczych. Szczególną uwagę poświęcono trzem zjawiskom anomalnym: *HPA Lag*, *Cold Start Latency* i *CPU Throttling Cascade*. Dla *HPA Lag* wska-

ziano ograniczenie metodologiczne: dostępne eksporty CSV pozwalają wiarygodnie opisać widoczny przebieg liczby replik i czas ich dołączania, ale nie pozwalają jednoznacznie wyznaczyć dokładnego momentu przekroczenia progu HPA dla każdej repliki.

6.2 Weryfikacja hipotezy badawczej

Hipoteza badawcza postawiona w rozdziale 1 brzmiała:

„Nie istnieje jedna uniwersalna metryka HPA; skuteczność strategii skalowania zależy od charakterystyki obciążeniowej aplikacji (workload profile). Błędny wybór metryki prowadzi do niedoskalowania lub przeskalowania.”

Wyniki eksperymentów **częściowo potwierdzają** postawioną hipotezę, jednak w sposób bardziej złożony niż pierwotnie zakładano.

HPA CPU okazał się jedyną skuteczną strategią; wywołał skalowanie zarówno dla aplikacji CPU-bound, osiągając 29 replik, jak i I/O-bound, z wynikiem 7 replik. Jest to wynik częściowo sprzeczny z hipotezą, która przewidywała nieskuteczność HPA CPU dla I/O-bound. W praktyce, przy 150 współbieżnych VUs, narzut *Uvicorn* i pętli zdarzeń *asyncio* generował wystarczające obciążenie CPU, by przekroczyć próg 50% liczony względem `requests.cpu`. Oznacza to, że HPA CPU *może* być skuteczny również dla aplikacji I/O-bound, pod warunkiem odpowiednio niskiej wartości `requests.cpu`, względem której HPA oblicza procentową użycie procesora, oraz wysokiej współbieżności. Jednak dla CPU-bound skuteczność ta była okupiona poważnym przeskalowaniem: HPA rozbudował system do 29 replik, niemal wyczerpując zdefiniowany limit `maxReplicas=30`, przy jedynie 26% wzroście przepustowości, co odpowiada efektywności $E = 0,01$.

HPA Memory był nieskuteczny w obu profilach, zgodnie z przewidywaniami. Zużycie pamięci w zaimplementowanych aplikacjach jest statyczne i nie koreluje z liczbą żądań. Wynik ten potwierdza, że HPA Memory nie nadaje się jako jedyna metryka dla aplikacji o stałym profilu pamięciowym, choć może być wartościowy dla systemów cache, kolejek komunikatów czy przetwarzania dużych plików.

HPA Custom RPS nie wywołał skalowania, co jest wynikiem *pozornie* sprzecznym z hipotezą, która przewidywała jego uniwersalną skuteczność. Przyczyną nie był jednak błędny wybór metryki, lecz niewłaściwa kalibracja progu: 100 RPS/Pod okazało się wartością zbyt wysoką dla obu serwisów (łącznie RPS wynosił około 65, co przy 1–2 replikach daje około 32–65 RPS/Pod). Wynik ten prowadzi do istotnego uściślenia hipotezy:

Skuteczność strategii HPA zależy nie tylko od wyboru metryki, ale również od kalibracji progu. Błędny dobór progu może uniemożliwić skalowanie nawet przy prawidłowo wybranej metryce; odwrotnie, prawidłowo dobrany próg może uczynić skuteczną metrykę, która teoretycznie nie powinna działać dla danego profilu obciążeniowego.

Niezależnie od konkluzji dotyczącej hipotezy głównej, analiza zjawisk anomalnych dostarcza ilościowych danych na temat ograniczeń autoskalowania reaktywnego w środowisku GKE. Obserwowany czas rozbudowy od pierwszego zwiększenia liczby replik do maksimum wyniósł około 90 s dla `cpu-cpu` oraz około 120 s dla `io-cpu`, natomiast *Cold Start Latency* pojedynczych Podów utrzymywało się na poziomie kilkunastu–kilkudziesięciu sekund. Oznacza to, że system nie reaguje w skali pojedynczych sekund, a proces skalowania musi być interpretowany względem faz ramp-up, fazy stabilnej oraz opóźnionego ramp-down.

6.3 Ograniczenia pracy

Należy wskazać następujące ograniczenia przeprowadzonych badań.

Aplikacje badawcze (`cpu-service`, `io-service`) symulują określone profile obciążeniowe, ale nie odzwierciedlają rzeczywistych systemów produkcyjnych, które często wykazują profile mieszane, łączące cechy CPU-bound i I/O-bound. Badanie ogranicza się do trzech metryk HPA (CPU, Memory i Custom RPS), nie analizując innych strategii, takich jak skalowanie na podstawie liczby żądań w toku (in-flight), metryk zewnętrznych (na przykład długości kolejki RabbitMQ), ani skalowania predykcyjnego opartego na modelach uczenia maszynowego. Wszystkie eksperymenty przeprowadzono w pojedynczej strefie GCP `eu-central2`, co oznacza, że nie badano wpływu opóźnień między strefami (cross-zone latency) ani scenariuszy multi-region. Progi HPA (CPU 50%, Memory 70%, RPS 100/Pod) zostały ustalone arbitralnie na podstawie wartości referencyjnych i nie były przedmiotem optymalizacji; jak wykazały badania, kalibracja progu ma kluczowe znaczenie dla skuteczności strategii. Rozszerzenie *SaaS Multi-Tenancy* (Faza VIII planu pracy) zostało zaprojektowane koncepcyjnie, ale nie zaimplementowane w kodzie, co oznacza, że wnioski dotyczące zachowania systemu w środowisku wielodostępnym pozostają hipotetyczne i wymagają dalszych badań.

6.4 Rekomendacje praktyczne

Na podstawie przeprowadzonych badań sformułowano następujące rekomendacje dla praktyków projektujących polityki autoskalowania w *Kubernetes*.

Fundamentalną zasadą jest dopasowanie metryki HPA do profilu obciążeniowego aplikacji oraz empiryczna kalibracja progu w warunkach zbliżonych do produkcyjnych. Jak wykazały badania, nawet prawidłowo dobrana metryka (Custom RPS) nie gwarantuje skalowania przy źle skalibrowanym progu, a metryka teoretycznie nieoptymalna (CPU) może okazać się skuteczna przy odpowiedniej konfiguracji limitów i progów.

W zakresie operacyjnym należy uwzględniać zjawiska anomalne przy definiowaniu SLA; *HPA Lag*, czas dołączania replik i *Cold Start Latency* oznaczają, że realistyczny czas reakcji systemu na nagły wzrost obciążenia jest liczony w dziesiątkach sekund lub minutach, a nie w pojedynczych sekundach. *CPU throttling* powinien być aktywnie monitorowany przez metrykę `container_cpu_cfs_throttled_seconds_total`, ponieważ nie powoduje awarii, ale drastycznie wydłuża czasy odpowiedzi, działając jako „cichy zabójca” wydajności. Dla testów obciążeniowych i środowisk produkcyjnych rekomenduje się bezpośrednią ekspozycję serwisów przez *GCP LoadBalancer* zamiast `port-forward`, co eliminuje wąskie gardło tunelowania przez *API server* i zapewnia realistyczne warunki sieciowe. Wreszcie, wdrożenie monitoringu przez *kube-prometheus-stack* przez Helm redukuje złożoność konfiguracji i zapewnia spójność wersji komponentów, co jest szczególnie istotne w środowiskach produkcyjnych.

6.5 Dalsze kierunki badań

Niniejsza praca otwiera kilka obiecujących kierunków dalszych badań.

Najpilniejszym rozszerzeniem jest implementacja modelu *SaaS Multi-Tenancy* (Faza VIII planu pracy) i przeprowadzenie testów w środowisku wielodostępnym na GKE. Zaprojektowana koncepcja przewiduje dwa poziomy izolacji: *application-level*, gdzie tenanci

są rozróżniani nagłówkiem *X-Tenant-ID* i współdzielą infrastrukturę, oraz *namespace-per-tenant*, gdzie każdy tenant otrzymuje dedykowaną przestrzeń nazw z własnymi *ResourceQuota*, *NetworkPolicy* i niezależnymi HPA. Środowisko GKE, z automatycznym skalowaniem węzłów i natywną integracją z ekosystemem GCP, jest naturalną platformą do tych badań, ponieważ eliminuje ograniczenia sprzętowe występujące na pojedynczym serwerze lokalnym i umożliwia testowanie z realistyczną liczbą tenantów. Kluczowymi pytaniami badawczymi są: czy HPA chroni „cichego” tenanta przed degradacją wydajności spowodowaną przez „głośnego sąsiada”, czyli noisy neighbor, czy zjawiska anomalne (*HPA Lag*, *Cold Start Latency*, *CPU Throttling Cascade*) nasilają się w środowisku wielodostępnym oraz jaki jest koszt infrastrukturalny izolacji *namespace-per-tenant* w porównaniu z modelem *application-level*.

Kolejnym, nowo zidentyfikowanym kierunkiem, wynikającym bezpośrednio z przeprowadzonych badań, jest systematyczna analiza wpływu kalibracji progów HPA na skuteczność skalowania. Jak wykazały eksperymenty, próg 100 RPS/Pod dla HPA Custom RPS okazał się zbyt wysoki; interesującym pytaniem badawczym byłoby wyznaczenie optymalnych progów dla każdej kombinacji profilu obciążeniowego i strategii HPA, potencjalnie z wykorzystaniem technik automatycznego strojenia (na przykład Bayesian optimization).

Dalszym kierunkiem jest rozszerzenie macierzy badawczej o dodatkowe profile obciążeniowe i strategię HPA. Włączenie *memory-service* jako trzeciego profilu (memory-bound) umożliwiłoby weryfikację, czy HPA Memory staje się skuteczne dla aplikacji, których zużycie pamięci koreluje z obciążeniem; jest to pytanie, na które obecne badania nie dają odpowiedzi. Analiza metryk takich jak liczba żądań w toku (*http_requests_inflight*) lub kombinacji wielu metryk w jednym HPA mogłaby ujawnić strategię pośrednie między prostotą HPA CPU a uniwersalnością HPA Custom RPS. Porównanie reaktywnego HPA z podejściami predykcyjnymi, na przykład opartymi na LSTM, które przewidują przyszłe obciążenie i skalują proaktywnie, mogłoby wskazać drogę do eliminacji problemu *HPA Lag*.

W warstwie infrastrukturalnej wartościowym uzupełnieniem byłyby analiza kosztowa różnych strategii HPA w modelu pay-as-you-go GCP, szczególnie w kontekście *multi-tenancy*, gdzie przeskalowanie (jak zaobserwowane 29 replik dla *cpu-cpu*) przekłada się bezpośrednio na wyższe koszty, oraz powtórzenie eksperymentów w konfiguracji multi-zone w celu zbadania wpływu opóźnień między strefami na skuteczność HPA i zjawiska anomalne. Docelowo, walidacja wniosków na rzeczywistych aplikacjach produkcyjnych (na przykład systemie rezerwacji czy przetwarzaniu obrazu w pipeline ML) pozwoliłaby ocenić, na ile wyniki uzyskane na syntetycznych serwisach badawczych przenoszą się na systemy klasy enterprise.

6.6 Podsumowanie końcowe

Niniejsza praca magisterska stanowi kompletne studium empiryczne nad skutecznością strategii *HorizontalPodAutoscaler* w *Kubernetes* dla różnych profili obciążeniowych, zrealizowane w całości w środowisku *Google Cloud Platform*. Główny wkład pracy obejmuje: systematyczną analizę literatury identyfikującą cztery luki badawcze, w tym nowo zidentyfikowaną lukę dotyczącą analizy zjawisk anomalnych w autoskalowaniu; projekt macierzy badawczej 2×3 umożliwiającej izolowane porównanie strategii HPA; implementację czterech mikroservisów o kontrolowanych profilach obciążeniowych wraz z pełną infrastrukturą monitorującą; automatyzację procesu badawczego, od wdrożenia po wykonanie testów i eksport danych; oraz koncepcję rozszerzenia *SaaS Multi-Tenancy* dla środowiska

GKE, uwzględniającą specyfikę zarządzanego klastra i mechanizmy izolacji *Kubernetes*.

Przeprowadzone badania empiryczne dostarczają dowodów na to, że skuteczność strategii HPA zależy zarówno od wyboru metryki, *jak i* od kalibracji jej progu. Głównym, nieprzewidzianym wynikiem jest to, że HPA CPU, a nie HPA Custom RPS, okazał się strategią najbliższą „uniwersalnej” w warunkach testowych: skalował się zarówno dla CPU-bound (29 replik, eliminacja błędów z 37,70% do 0,04%), jak i dla I/O-bound (7 replik, wzrost przepustowości o 59%). Jednocześnie ujawniono poważny problem przeskalowania: 29 replik przy limicie `maxReplicas=30`, co oznacza niemal całkowite wyczerpanie marginesu bezpieczeństwa, oraz fundamentalne znaczenie kalibracji progu (HPA Custom RPS nie skalował się nie dlatego, że metryka jest nieskuteczna, ale dlatego, że próg 100 RPS/Pod był zbyt wysoki). Badania dostarczają również ilościowej charakterystyki zjawisk przejściowych: rozbudowa od pierwszego zwiększenia liczby replik do maksimum trwała około 90–120 s, a *Cold Start Latency* pojedynczych Podów utrzymywało się w zakresie kilkunastu–kilkudziesięciu sekund. Wyniki te mają istotne znaczenie praktyczne dla projektantów systemów *SaaS* działających na *Kubernetes* w chmurze.

Otwarte, odtwarzalne środowisko badawcze dostępne w repozytorium umożliwia niezależną weryfikację wyników i stanowi fundament dla dalszych badań w kierunku *multi-tenancy*, optymalizacji progów i skalowania predykcyjnego.

Bibliografia

- [1] B. Brazil. *Prometheus: Up & Running*. O'Reilly Media, 2018.
- [2] B. Burns, J. Beda, K. Hightower. *Kubernetes: Up and Running*. O'Reilly Media, wydanie 3, 2023.
- [3] E. Casalicchio, V. Perciballi. Auto-scaling of containers: The impact of relative and absolute metrics. *2017 IEEE 2nd International Workshops on Foundations and Applications of Self* Systems (FAS*W)*, strony 207–214. IEEE, 2017.
- [4] Google Cloud. Artifact registry documentation. Raport instytutowy, Google Cloud, 2024.
- [5] Google Cloud. Cloud load balancing documentation. Raport instytutowy, Google Cloud, 2024.
- [6] Google Cloud. Compute engine documentation. Raport instytutowy, Google Cloud, 2024.
- [7] Google Cloud. Google kubernetes engine documentation. Raport instytutowy, Google Cloud, 2024.
- [8] Grafana Labs. Grafana k6 documentation. Raport instytutowy, Grafana Labs, 2024.
- [9] Helm Authors. Helm — the package manager for kubernetes. Raport instytutowy, Cloud Native Computing Foundation, 2024.
- [10] B. Ibryam, R. Huß. *Kubernetes Patterns*. O'Reilly Media, 2021.
- [11] Kubernetes SIG Instrumentation. Prometheus adapter — custom metrics api server. Raport instytutowy, Kubernetes, 2024.
- [12] S. Newman. *Building Microservices*. O'Reilly Media, wydanie 2, 2021.
- [13] N. Poulton. *Docker Deep Dive*. Leanpub, 2023.
- [14] Prometheus Community. kube-prometheus-stack helm chart. Raport instytutowy, Prometheus Community Helm Charts, 2024.
- [15] Prometheus Community. Prometheus operator documentation. Raport instytutowy, Cloud Native Computing Foundation, 2024.
- [16] S. Ramírez. Fastapi — modern, fast (high-performance) web framework. Raport instytutowy, FastAPI, 2024.
- [17] The Kubernetes Authors. Horizontal pod autoscaler. Raport instytutowy, Kubernetes Documentation, 2024.

Spis rysunków

5.1	Przebieg skalowania dla scenariuszy CPU-bound — porównanie strategii HPA	29
5.2	Przebieg skalowania dla scenariuszy I/O-bound — porównanie strategii HPA	29
5.3	Zużycie CPU dla każdej widocznej repliki w scenariuszu <code>cpu-cpu</code>	30
5.4	Zużycie CPU dla każdej widocznej repliki w scenariuszu <code>io-cpu</code>	30
5.5	Scenariusze bazowe (bez HPA) — liczba replik w czasie	31

Spis tabel

2.1	Podsumowanie stosu technologicznego projektu wraz z rolami poszczególnych komponentów w systemie badawczym	12
3.1	Macierz badawcza 2×3 — identyfikatory scenariuszy testowych	13
3.2	Zasoby przydzielone serwisom	16
5.1	Parametry scenariuszy bazowych	26
5.2	Parametry scenariuszy CPU-bound	26
5.3	Parametry scenariuszy I/O-bound	26
5.4	Wyniki scenariuszy bazowych	27
5.5	Wyniki scenariuszy CPU-bound	27
5.6	Wyniki scenariuszy I/O-bound	28
5.7	Efektywność skalowania — porównanie strategii	34